Volume VI

Engineering Chronology & Development Logs

CreativeWorkzStudio LLC / ProfessorSeanEX

Version 0.2.0-dev, 2026-11-12

Table of Contents

1. DevLog 001: Genesis, Paradigm Pivot & Invariants . . . 2
   1.1. Abstract . . . 2
   1.2. Project Genesis: Clean Foundations and the 5-Block Standard . . . 2
   1.3. The Early Prototype: The Survival-Battery Fallacy . . . 2
   1.4. The Paradigm Pivot: One Clarification That Changed Everything . . . 2
   1.5. Dual-Frontend Architecture: Native X11 & Terminal CLI . . . 3
   1.6. Why Game Development Testing Differs from Software Testing . . . 4
   1.7. What Lies Ahead . . . 4
2. DevLog 003: Central Manifest & Pre-Flight Spec . . . 5
   2.1. Abstract & Engineering Intent . . . 5
   2.2. The Core Problem: Loose Bolts & Silent Corruption . . . 5
   2.3. Architectural Implementation . . . 5
   2.4. 3. "Expected" vs. "Actual" Gap Analysis . . . 6
   2.5. Summary & Next Steps . . . 6
3. DevLog 004: Pedagogical Metadata & Active DAG Contracts . . . 7
   3.1. Abstract & Engineering Intent . . . 7
   3.2. The Pedagogical Balancing Act: Beginner Mission vs. Senior Contracts . . . 7
   3.3. Temporal Precision: Mandating Full ISO 8601 Calendar Dates . . . 8
   3.4. Active DAG Dependency Contracts: Turning Comments into Machine Tripwires . . . 8
   3.5. Subsystem Dependency Manifest . . . 9
   3.6. Empirical Verification & Test Results . . . 10
   3.7. Conclusion & Next Horizons . . . 11
4. DevLog 005: Ontological Sub-Assemblies & 80-Column Discipline . . . 12
   4.1. Abstract & Engineering Intent . . . 12
   4.2. The Need for IDE-First Hover Documentation . . . 12
   4.3. 1.1 The Clangd Hover Contract . . . 12
   4.4. The Machine Metaphor: From Cosmos to Fasteners . . . 12
   4.5. Visual Border & Delimiter Standards . . . 13
   4.6. Chronological Reorganization of Developer Logs . . . 14
   4.7. Empirical Verification Telemetry . . . 14
   4.8. Summary . . . 15
5. DevLog 006: Source Implementation & Root Pedagogical Commenting . . . 16
   5.1. Executive Summary . . . 16
   5.2. Architectural Intent: End-to-End Documentation Consistency . . . 16
   5.3. Source Code Implementation Matrix . . . 16
   5.4. Root Configuration Pedagogical Enhancement . . . 17
   5.5. Strict 80-Column Verification Audit . . . 18
   5.6. Verification & Telemetry Results . . . 18
6. DevLog 007: Archival Extraction & The 2017–2018 Demo Baseline . . . 19
   6.1. Abstract . . . 19
   6.2. Historical Context: Rediscovery of the Genesis Datasets . . . 19
   6.3. The Extraction Toolchain: `docs/tools/extract_demo_data.py` . . . 19
   6.4. The 10-Volume Baseline Documentation Compendium . . . 20
   6.5. C11 Engine Rebuilding Roadmap . . . 20

7. DevLog 008: Canonical Skills, The Dreamscape & Ground-Truth Pacing . . . 22
7.1. Abstract . . . 22
7.2. Lessons Learned: Game Developer vs. Software CRUD Thinking . . . 22
7.3. Implementation Telemetry & Invariant Verification . . . 22
8. DevLog 009: Macro-History, Thematic Triad & Multi-Ending Architecture . . . 24
8.1. Abstract . . . 24
8.2. Macro-Historical Canon: The 3,000-Year Dynastic Wave . . . 24
8.3. The Thematic Triad: "Trauma is Inherited, Destiny is Chosen" . . . 24
8.4. Ludological Architecture & Multi-Ending Specification . . . 25
8.5. Verification Telemetry . . . 25
9. DevLog 010: Layer 0 Subsystem Modularization & Data-Driven Architecture . . . 26
9.1. Executive Summary . . . 26
9.2. Architectural Motivation & The Problem of Flat Foundations . . . 26
9.3. Subsystem Stratification & Pathing Standards . . . 27
9.4. Revision 2 Filesystem & JSON Engines . . . 27
9.5. External Data Menus & Fallback Tripwires . . . 28
9.6. Verification & Telemetry Results . . . 28
9.7. Conclusion & Architectural Reflection . . . 29
10. DevLog 011: Four-Tier Verification Strata & Ontological Testing Framework . . . 30
10.1. Executive Summary . . . 30
10.2. Architectural Motivation & The Problem of Amorphous Testing . . . 30
10.3. The Four-Tier Testing Strata . . . 31
10.4. The Four Ontological Testing Sections . . . 32
10.5. Preparing for the Interactive Debugger . . . 32
10.6. Verification Telemetry & Verification Matrix . . . 32
11. DevLog 012: Interactive Debugging System & State Rewind Architecture . . . 34
11.1. Executive Summary . . . 34
11.2. Architectural Motivation & The Problem of Blind Mutation . . . 34
11.3. Decoupled State Snapshot Ring Buffer Architecture . . . 34
11.4. Dynamic Breakpoints & Differential State Analysis . . . 35
11.5. Data-Driven Debugger Profiles & Fallback Monoliths . . . 36
11.6. Verification Telemetry & Verification Expansion . . . 36
12. DevLog 013: Morality Architecture, Bystander Corridor & Ten-Ending Matrix . . . 37
12.1. Abstract . . . 37
12.2. Decoupling Morality ("The Why") from Elemental Expression ("The How") . . . 37
12.3. Psychological & Systemic Catalysts: The Bystander Corridor vs. The Butterfly Effect . . . 37
12.4. Complete Taxonomy of the Moral Sway Continuum ([-100, +100]) . . . 38
12.5. The 10-Ending Matrix & The Witness Observer . . . 39
12.6. Diegetic Item Architecture & Save Mechanics . . . 39
12.7. Historical Grounding: The 2017–2018 Demo Baseline . . . 40
12.8. Verification & Invariant Audit . . . 40
13. DevLog 014: High Fantasy Presentation Engine & 5-Screen UI . . . 41
13.1. Executive Summary . . . 41
13.2. Visual Balance Dilemma & High-DPI Typography Subsystem . . . 41
13.3. Responsive Widescreen Scaling & Isometric Dais Zoom . . . 42
13.4. Dedicated 5-Screen Navigation Architecture . . . 42

13.5. Architectural Review & State of the Engine (v1.4.0) . . . 43
13.6. Verification Telemetry . . . 44
13.7. Summary & Forward Stance . . . 45
14. DevLog 015: The Big Machine, Master Turn Gear & Core Subsystem Elevation . . . 46
14.1. Executive Summary . . . 46
14.2. The Architectural Paradigm: The Big Machine & The Master Turn Gear . . . 46
14.3. Layer 0 Modernization: The Complete Four-Strata Engine Bedrock . . . 47
14.4. The Cog Elevation & Orchestrator Protocol . . . 48
14.5. Four-Strata Verification Ontology & 51/51 CTest Pass Proof . . . 48
14.6. Telemetry & Pre-Flight Diagnostic Self-Audit . . . 49
14.7. The Fractal 5-Block Invariant: Code, Manifests, Tests & Git History . . . 50
14.8. Architectural Next Steps & Horizon Ahead . . . 50
15. DevLog 016: Native Git Commit Validation, Audio Elevation & Movie Reel . . . 51
15.1. Milestone Scope & Commit Progression . . . 51
15.2. Enforcing 5-Block Provenance at the Commit Boundary (36427ba) . . . 51
15.3. Elevating audio/ to Layer 0 & Building the Movie Reel (9c209ad, 52149a3) . . . 52
15.4. Verification & Cross-Tier Audiovisual Parity (52149a3) . . . 52
15.5. Architectural Takeaway & Next Horizon . . . 52
16. DevLog 017: Orthogonal 35 Classes, 28 Jobs, Generational War & 4D Dream Clock . . . 53
16.1. Resolving 2017–2018 Demo & Early Documentation Ambiguities . . . 53
16.2. Orthogonal Character Engine: 35 Classes & 28 World Jobs . . . 53
16.3. Adolescent Crucible, Poisoned Mandate & Two-Pass Grand Tour . . . 54
16.4. The 4D "Dream Clock" Temporal Registration Engine . . . 54
16.5. Layer 1 Zero-Heap Contract Verification . . . 54
17. DevLog 018: Standalone Engine Config Decoupling & Rev 5 Block 1/2 Standards . . . 55
17.1. Separating Engine Mechanics (engine/config/) from Game Lore (game/data/) . . . 55
17.2. Movement-Resilient Metadata & Unified Block 2 Opening Contracts . . . 55
17.3. Validator Modularization & Header Contract Verification . . . 56
17.4. Readiness for Tier-0 BIOS & .rodata ROM Vaults . . . 56
18. DevLog 019: BIOS/ROM Vaults, AWAIT Probes & The C23 Fall-Down Ladder . . . 57
18.1. The Autonomous Root Spine: log/ + debug/ + io/ . . . 57
18.2. 2. "Engine-as-a-Java" Portability: The C23 Fall-Down Ladder . . . 57
18.3. Crude-to-Fine Verification Across log/, debug/, and io/ (tests/) . . . 58
18.4. Next Horizon: Documenting the Root Tree & Auditing core/ . . . 59
19. DevLog 020: 5-Block AsciiDoc, Live Code Snapshots & Paired Textbooks . . . 60
19.1. Textbook-Grade Live Code Snapshots (docs/code_snapshots_manifest.json) . . . 60
19.2. The 5-Block AsciiDoc Comment Standard & --validate-layout . . . 60
19.3. Paired Code Folders, Architectural Chronicles & Linked Offline Series . . . 61
19.4. Verification Summary Across Code, Layout & Books . . . 61
19.5. Synchronizing the Repository & Preparing for core/ Audits . . . 61
20. Developer Log 002: The Sacyra Chronicles, Dual Protagonists & TRPG Evolution . . . 62
20.1. Abstract . . . 62
20.2. Historical Provenance: The 2014–2018 High School Genesis . . . 62
20.3. Narrative Realignment: The Kingdom of Sacyra . . . 62
20.4. Ludological Evolution: The Tactical Strategy RPG (TRPG) . . . 63
20.5. Next Milestone Roadmap . . . 64

Appendix A: Wholesale Source Code Reference Plates 65
A.1. Pre-Flight Loose-Bolts Engine Walk (manifest_run_preflight) 65



Volume VI Synopsis

Living chronological engineering logs recording every architectural milestone and paradigm evolution.

Chapter 1. DevLog 001: Genesis, Paradigm Pivot & Invariants

Source File: `docs/dev_logs/2026/10/architecture/devlog_001_genesis_and_pivot.adoc`

1.1. Abstract

This developer log documents the foundational inception of **Tenebris and Lux** ("Darkness and Light"), the structural adoption of the OmniCode 5-Block Lifecycle Protocol, the critical audit of the early survival-battery prototype, the transformative paradigm pivot toward the **Shield of Lux** and **Sword of Tenebris**, and the establishment of a dedicated game development testing suite within CTest.

1.2. Project Genesis: Clean Foundations and the 5-Block Standard

Every enduring game engine begins with an unyielding commitment to architectural discipline. **Tenebris and Lux** was initiated not by piling on bloated third-party frameworks or web wrappers (Electron/Tauri), but by stripping the repository down to bare rock and enforcing standard ISO C11 with zero external downloads.

Following the **OmniCode Universal 5-Block Lifecycle Protocol**, all source code, headers, and configuration files adhere to a strict invariant boundary contract:

1. **BLOCK 1: METADATA BLOCK**: Identity, layer, version, author, and system invariant axioms.
2. **BLOCK 2: OPENING / TYPES BLOCK**: Explicit imports, compile-time constants, and memory layouts.
3. **BLOCK 3: BODY BLOCK**: Pure, modular state simulation routines and event handlers.
4. **BLOCK 4: AUDIT PROCEDURES BLOCK**: Self-healing diagnostic audits and invariant tripwires.
5. **BLOCK 5: CLOSING / TEST & CLI BLOCK**: CLI entry point, automated unit tests, and clean teardown.

1.3. The Early Prototype: The Survival-Battery Fallacy

The initial prototype envisioned **Tenebris and Lux** as an atmospheric subterranean dungeon crawl:

- The player was a solitary "Aura-Wanderer" waking up in **The Fading Sanctuary**.
- Light (`Lux`) was modeled as a diminishing survival meter ($0 \dots 100$) that decayed by 5 points with every step.
- Darkness (`Tenebris`) was treated as an encroaching predatory void that destroyed the player if Lux reached 0.
- Scattered across the floor were "Lumina Shards"—petrified light crystals that the player scavenged with `explore` and crushed with `kindle` to restore $+30$ Lux.

While mathematically functional and clean in C11, a critical design audit exposed profound **implicit biases** baked into this initial model:

- **Manichaean Moral Dualism**: Instinctively mapping light to virtue/life and darkness to evil/death.
- **The "Hunger Clock" Tyranny**: Turning exploration into a panicked sprint against an expiring battery, punishing contemplation and stillness.
- **Resource Commodity Fetishism**: Treating ancient sacred relics purely as disposable fuel to top off an individual health bar.

1.4. The Paradigm Pivot: One Clarification That Changed Everything

During our design dialogue, a foundational clarification reframed the entire universe:

> "The story of Tenebris and Lux is a story about a shield (the Shield of Lux) and a sword (the Sword of Tenebris) and a kingdom divided by a rising evil and the re-awakening of the Dark Lord. Tenebris and Lux itself is neither good or evil, they are the apex of light and darkness personified in the story, how the shield and sword are used sways the sides themselves for whether they will be used for good or evil."
>
> — Project Creator Directive

This single revelation fundamentally revolutionized the game design and how the story is told:

1.4.1. From Passive Attrition to Active Agency

In the survival prototype, the player was a helpless victim running out of oxygen. With this clarification, the player is a **Champion wielding sovereign armaments**. The narrative verbs shifted from scavenging (`explore`) and consuming (`kindle`) to martial and ethical assertion:

- `ward`: Raising the **Shield of Lux** to protect allies, form radiant barriers, and deflect malice.
- `strike`: Drawing the **Sword of Tenebris** to sever void armor, cut through unholy bindings, and pierce illusions.
- `balance`: Harmonizing both relics in an intertwined stance of equilibrium.

1.4.2. Disentangling Darkness from the Villain

Darkness is no longer the enemy. **Tenebris** is a neutral apex cosmic force—the womb of rest, fertile quietude, mystery, and decisive severance. The true villain is **The Dark Lord** (the ancient Libiran sovereign Usuru), a fallen king who broke the ancient seals to twist both light and shadow to his will.

1.4.3. The Moral Sway Continuum

Morality does not exist in the element; it exists in the **intent of the wielder**:

- **Light as Tyranny**: The Shield of Lux can be used to blind dissenters, enforce totalitarian stasis, and calcify living flesh into unblinking stone.
- **Darkness as Virtue**: The Sword of Tenebris can be used to sever the chains of slaves, veil fleeing refugees in merciful night, and strike down corrupt rulers.
- The game tracks `moral_sway` on a bounded continuum ($-100 \dots +100$), reflecting whether the bearer leans toward **Virtue & Mercy** or **Tyranny & Ruin**.

1.5. Dual-Frontend Architecture: Native X11 & Terminal CLI

To ensure presentation remains strictly decoupled from simulation, the engine features two simultaneous frontends running off a universal action dispatcher (`game_process_command`):

1. **Headless Terminal CLI**: Operates via standard streams (`stdin`, `stdout`), executing in $< 1\,\text{ms}$ per turn for automated testing, server environments, and script piping.
2. **Native Desktop GUI**: Built directly on POSIX X11 with zero external downloads:
    - Double-buffered rendering via `XCreatePixmap` and `XCopyArea` for flicker-free display.

- 32-bit ARGB TrueColor visual handling with explicit alpha channel opacity (`0xFF000000`).
- Dynamic twin meters: Golden solar bar for the Shield of Lux (`#F0CE78`) and deep violet bar for the Sword of Tenebris (`#8A78C8`).
- Tactical hotkeys and interactive widgets.

1.6. Why Game Development Testing Differs from Software Testing

Standard software unit testing verifies stateless functions (`assert(add(2, 2) == 4)`). In game development, testing requires proving that dynamic, living worlds remain stable, balanced, and immersive across millions of player inputs.

Our CTest testbed introduces two dedicated game dev testing suites:

1.6.1. Game State Invariant Testing (`tests/test_game_invariants.c`)

- Mathematical bounds verification: Shield power [0..100], Sword edge [0..100], and Moral Sway [-100..+100] never overflow or underflow under continuous spamming.
- Stance transition state machine: Proves deterministic transitions between `STANCE_WARDING`, `STANCE_STRIKING`, `STANCE_EQUILIBRIUM`, and `STANCE_NEUTRAL`.
- Topological pathway validation across Kingdom provinces.
- Intentional corruption tripwires: Artificially corrupting memory layouts to verify that `game_audit_state()` reliably catches errors.

1.6.2. Gameplay Scenarios & Replay Testing (`tests/test_gameplay_scenarios.c`)

- **Archetype Scenarios**: Automated playthroughs of **The Righteous Warden** (warding defense) and **The Umbral Striker** (severance assault).
- **Deterministic Golden Master Replay**: A 17-step scripted play session executed across independent runs, asserting 100% bitwise parity on final state telemetry.
- **Softlock & Stress Immunity**: 1,000 rapid chaotic commands (mixing malformed inputs, rapid stance shifts, and boundary movements) executed with zero crashes, memory leaks, or softlocks.
- **Simulation Latency Benchmark**: 10,000 turn dispatches executed in **160.29 ms (16.02 μs per turn)**—consuming less than 0.1% of a 60 FPS ($16.6\,\text{ms}$) frame budget.

1.7. What Lies Ahead

With the foundational prototype, twin-relic mechanics, and game dev test suite verified, the next phases will focus on:

- Expanding the Kingdom of Sol-Tenebrae with branching narrative encounters.
- Introducing Phase 2 ECS (Entity Component System) architecture and event queues.
- Developing Phase 3 2D/2.5D ambient lighting software rasterizers.

Chapter 2. DevLog 003: Central Manifest & Pre-Flight Spec

Source File: `docs/dev_logs/2026/10/architecture/devlog_003_preflight_and_manifest_architecture.adoc`

2.1. Abstract & Engineering Intent

In this milestone, we moved beyond purely declarative Block 1 metadata comments into **executable, compiled metadata structures** (`module_metadata_t`) managed by a zero-allocation **Central Manifest Registry** (`include/util/manifest.h`, `src/util/manifest.c`). Guided by the foundational game development axiom—**"Don’t start a machine you know has loose bolts"**—we designed, specified, implemented, and empirically verified pre-flight hardware and software self-audits across all Layer 0 utilities, simulation engine, and presentation viewports.

2.2. The Core Problem: Loose Bolts & Silent Corruption

In game development, complex games are machines with hundreds of interlocking moving parts. A subtle defect in a foundational layer—such as a non-monotonic clock, an integer overflow on damage saturation, an unchecked negative rectangle dimension, or an unbuffered string copy—can propagate upward, manifesting as elusive micro-stutters, physics drift, state desynchronization, or catastrophic runtime crashes.

Rather than waiting for the game loop to execute before identifying structural anomalies, the engine must perform systematic pre-flight verification on startup, classifying faults along a graded severity spectrum: * **Warnings (`SEVERITY_WARNING`)**: Log notices (e.g. coarse sleep timer resolution); launch proceeds normally. * **Critical Faults (`SEVERITY_CRITICAL`)**: Subsystem offline, but engine core intact (e.g. no `$DISPLAY` for X11 GUI); downgrade gracefully to safe mode (terminal CLI). * **Fatal Violations (`SEVERITY_FATAL`)**: Core invariants broken (e.g. non-monotonic time, arithmetic bounds failure, corrupt initial state); halt the engine immediately with a detailed diagnostic manifest.

2.3. Architectural Implementation

2.3.1. Compiled Metadata Structures

Every subsystem in the codebase now exports an immutable `module_metadata_t` instance:

```
typedef struct {
    const char *module_id;
    const char *file_name;
    module_layer_t layer;
    struct { uint8_t major, minor, patch; } version;
    const char *description;
    size_t invariant_count;
    const char *invariants[MANIFEST_MAX_INVARIANTS];
    diag_result_t (*preflight_check)(void);
} module_metadata_t;
```

2.3.2. Subsystem Integrations

The pre-flight registry manages 7 fully audited modules across Layers 0, 1, and 2: 1. `core.util.manifest`: Self-audits registry capacity limits, duplicate registration rejection, and severity escalation rules. 2. `core.util.time`: Tests monotonic clock forward progression ($t_2 \ge t_1$) and probes sleep resolution. 3. `core.util.math`: Tests clamp bounds, saturating add/sub overflow safety, division-by-zero defense, and nudge directionality. 4. `core.util.geom`: Tests negative dimension clamping, boundary point containment, AABB overlap reflexivity, and viewport bounds containment. 5. `core.util.string`: Tests buffer safe truncation null-termination, NULL pointer resilience, and whitespace trimming edge cases. 6. `sim.engine.core`: Tests initial relic state integrity, resonance saturation ceilings, deterministic stance state transitions, and diagnostic tripwires on corrupt states. 7. `gui.viewport.x11`: Tests button layout non-overlap and asserts

window bounds (`gui_audit_layout`), while checking X11 display availability (`gui_is_available`). If `$DISPLAY` is unconnectable, emits `SEVERITY_CRITICAL` to trigger automatic CLI safe mode fallback.

2.4. 3. "Expected" vs. "Actual" Gap Analysis

Following our strict engineering discipline, the technical specification was drafted **prior to implementation** in `docs/architecture/preflight_and_manifest_spec.adoc`. Upon completion, empirical benchmarks were gathered and recorded:

Verification Item	Expected Baseline	Actual Result	Status
Module Registration	≥ 5 Core Modules	**7 Modules Registered**	**EXCEEDED**
Severity Spectrum	4 Discrete Levels	**5 Graded Levels** (`PASS`, `INFO`, `WARN`, `CRIT`, `FATAL`)	**EXCEEDED**
Pre-Flight Telemetry	80-Column ASCII Table	**Integrated Table** with status badges and policy verdicts	**SATISFIED**
CTest Integration	6/6 Suites Passing in $< 0.05\,\text{s}$	**6/6 Passing in $0.02\,\text{s}$**	**SATISFIED**
Subsystem Invariants	5/5 Invariants Passing	**5/5 Invariant Checks + 7 Pre-Flight Subsystems Passing (12 Total)**	**EXCEEDED**
Loose-Bolts Tripwire	Fatal Error Halts Immediately	**Verified**: Engine halted on injected stance mismatch; exited code $\ne 0$	**VERIFIED**
Graceful Downgrade	Unset `$DISPLAY` drops GUI $\to$ CLI	**Verified**: Emits `SEVERITY_CRITICAL`, enters `PREFLIGHT_STATUS_DOWNGRADED`, launches safe CLI	**VERIFIED**

2.5. Summary & Next Steps

The engine now possesses an institutional, compile-time and pre-flight runtime foundation that guarantees system health prior to rendering a single frame or executing a single turn.

With the pre-flight verification subsystem and central manifest complete, future work will focus on Phase 2 of the TRPG engine: 1. Laying out the 2D discrete tactical battlefield grid. 2. Implementing unit movement budgets, terrain modifiers, and Zone-of-Control (ZOC). 3. Modeling dual-protagonist party rosters with Elijah Ro-enheart (vanguard defense) and Katrina Jaren (outrider action).

Chapter 3. DevLog 004: Pedagogical Metadata & Active DAG Contracts

Source File: docs/dev_logs/2026/10/architecture/devlog_004_pedagogical_metadata_and_dependency_contracts.adoc

3.1. Abstract & Engineering Intent

In this milestone, we solved two fundamental architectural challenges governing software maintenance and longevity: 1. **The Pedagogical Metadata Dilemma**: How to structure source file metadata so that a novice programmer never feels overwhelmed or lost, while an experienced systems architect never feels patronized or deprived of rigorous technical contracts. 2. **Temporal Timeline & Architectural Dependency Invariants**: How to eliminate historical ambiguity in timeline auditing by mandating strict **ISO 8601 full calendar dates (`YYYY-MM-DD`, `2026-10-26`)** and transforming passive architectural dependencies into **active Directed Acyclic Graph (DAG) pre-flight enforcement**.

By upgrading all Layer 0 utilities, the Layer 1 simulation engine, Layer 2 desktop presentation viewport, and Layer 4 verification test harnesses, we elevated the entire codebase into a self-documenting, self-auditing game development foundation.

3.2. The Pedagogical Balancing Act: Beginner Mission vs. Senior Contracts

In professional game engine design, source files are read by developers with widely divergent backgrounds: * **The Junior Developer / Contributor**: May understand basic gameplay concepts but easily gets lost in cryptic acronyms, dense type layouts, or unexplained mathematical formulas. If a header only contains bare function signatures without context, they do not understand **why** the module exists or what breaks if they alter a value. * **The Senior Systems Engineer / Architect**: Needs instant, unambiguous answers regarding memory allocations (heap vs. stack), concurrency reentrancy, cache latency budgets, interface ABI stability, and upstream prerequisites. If a file contains verbose narrative hand-waving without concrete invariants, they cannot verify safety guarantees or integration boundaries.

3.2.1. The Dual-Audience Protocol

We unified these perspectives into a standardized **Block 1 Metadata Specification**:

```
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Identity, Provenance, Invariant Boundary Contracts & System Anchors )
// ============================================================================
// @project       Tenebris and Lux
// @file          time_utils.h
// @id            core.util.time
// @layer         Layer 0: Core Utilities
// @version       1.0.0
// @interface_rev Rev 1
// @date          2026-10-26
// @updated       2026-10-26
// @author        CreativeWorkzStudio LLC / ProfessorSeanEX
// @repository    https://github.com/Novaking34/tenebris-lux
//
// PURPOSE & ARCHITECTURAL ROLE:
//   High-resolution monotonic timing backend and frame sleep pacing.
//   Serves as the foundation for elapsed time measurement across simulation
//   ticks and presentation frame pacing (60 FPS latency budget).
//
// DEPENDENCY & RESOURCE CONTRACTS:
//   - Upstream Dependencies: None (Foundational leaf module).
//   - Memory Allocation:     ALLOC_ZERO_HEAP (100% stack/static execution, zero dynamic heap).
```

```
//   - Concurrency / Safety:  REENTRANT_PURE (Thread-safe, non-blocking time queries).
//   - Latency Budget:        < 5 microseconds per query.
//
// AXIOMATIC INVARIANTS:
//   1. Monotonicity: Time guarantees strictly non-decreasing clock ticks (t2 >= t1).
//   2. POSIX conformance: CLOCK_MONOTONIC timing source without drift or leap adjustments.
//   3. Frame sleep pacing: Non-negative sleep durations (negative inputs clamped to 0).
// ============================================================================
```

- **For Beginners**: The `PURPOSE & ARCHITECTURAL ROLE` field clearly frames the module's place in the universe using plain English, explaining both its simulation role and what other systems rely upon it.
- **For Seniors**: The `DEPENDENCY & RESOURCE CONTRACTS` and `AXIOMATIC INVARIANTS` fields supply exact, uncompromised technical contracts: zero heap allocation (`ALLOC_ZERO_HEAP`), pure reentrancy (`REENTRANT_PURE`), sub-5-microsecond latency, and mathematical clamping properties.

3.3. Temporal Precision: Mandating Full ISO 8601 Calendar Dates

Prior iterations utilized a compact month-and-year epoch stamp (e.g. `2026.10`). While concise, empirical timeline auditing revealed that month/year stamps obscure chronological sequencing: 1. When multiple architectural refactorings occur across consecutive days within the same month, month-level stamps fail to convey sequence. 2. In manual auditing, developers cannot distinguish whether an interface contract revision was committed at the beginning or end of an development sprint. 3. Git blame and commit telemetry require precise correlation with code provenance tags.

Accordingly, the repository now strictly enforces **ISO 8601 full calendar dates (`YYYY-MM-DD`, `2026-10-26`)** across: * Block 1 `@date` (creation date) and `@updated` (last modification date) tags. * Compiled `module_metadata_t.last_updated` string constants. * Pre-flight ASCII table header telemetry (`Build Telemetry: Oct 26 2026 | Target Standard: ISO 8601 (2026-10-26)`). * All technical architecture specifications and dev logs.

3.4. Active DAG Dependency Contracts: Turning Comments into Machine Tripwires

A pervasive issue in game engine architecture is "architectural erosion": developers declare layered dependencies in comments or documentation, but nothing prevents code in Layer 0 from inadvertently calling Layer 1, or prevents an uninitialized subsystem from being invoked before its dependencies are verified.

We elevated dependencies from passive comments into an **Active Directed Acyclic Graph (DAG)** validated at boot time.

3.4.1. Compiled 5-Dimension Metadata Schema

In `include/util/manifest.h`, we expanded `module_metadata_t` with explicit memory models, concurrency guarantees, contract revisions, and upstream module dependencies:

```
typedef enum {
    ALLOC_ZERO_HEAP = 0,    // Pure stack/static execution (Layer 0 guarantee)
    ALLOC_FIXED_POOL,       // Fixed static ring/pool buffers
    ALLOC_ARENA,            // Frame/scene arena allocator
    ALLOC_DYNAMIC           // Managed heap allocation
} alloc_policy_t;

typedef enum {
    CONCURRENCY_REENTRANT = 0, // Thread-safe, stateless pure functions
```

```
    CONCURRENCY_THREAD_LOCAL,  // Per-thread state
    CONCURRENCY_MAIN_THREAD    // Viewport/GUI event loop affinity
} concurrency_policy_t;

typedef struct {
    /* 1. Identity & Provenance */
    const char *module_id;          // e.g. "sim.engine.core"
    const char *file_name;          // e.g. "game.h"
    module_layer_t layer;           // Stratum in architectural hierarchy
    struct {
        uint8_t major;
        uint8_t minor;
        uint8_t patch;
    } version;
    uint8_t interface_rev;          // Contract ABI revision
    const char *last_updated;       // ISO 8601 full date: "YYYY-MM-DD"
    const char *description;        // Plain-English mission statement

    /* 2. Architectural Dependencies */
    size_t dependency_count;
    const char *dependencies[MANIFEST_MAX_DEPENDENCIES]; // Upstream module IDs required

    /* 3. Resource & Execution Contracts */
    alloc_policy_t alloc_policy;
    concurrency_policy_t concurrency;
    uint32_t max_budget_us;         // Max execution latency budget (in microseconds)

    /* 4. Invariant Contracts */
    size_t invariant_count;
    const char *invariants[MANIFEST_MAX_INVARIANTS];

    /* 5. Pre-Flight Machine Self-Audit */
    diag_result_t (*preflight_check)(void);
} module_metadata_t;
```

3.4.2. Pre-Flight DAG Enforcement Algorithm

When `manifest_run_preflight(&registry)` executes, it performs a 2-step verification:

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│ STEP 1: Upstream DAG Dependency Graph Validation                                             │
│ For each registered module:                                                                  │
│   For each declared dependency in dependencies[]:                                            │
│     Verify upstream module_id is registered in manifest.                                     │
│     If missing -> Flag SEVERITY_FATAL, HALT BOOT IMMEDIATELY                                 │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
                                                 │
                              | (All Upstream Dependencies Satisfied)
                              ▯
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│ STEP 2: Functional Loose-Bolts Subsystem Pre-Flight Checks                                   │
│ Execute preflight_check() function pointer per module.                                       │
│ Apply graded policy escalation:                                                              │
│   PASS -> OK | WARN -> Warnings | CRIT -> Downgrade | FATAL                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```

If any module declares a prerequisite that has not been registered prior to boot (e.g. `sim.engine.core` declaring `core.util.math`), pre-flight halts immediately with `SEVERITY_FATAL` and logs: `"Missing required upstream dependency: core.util.math"`.

3.5. Subsystem Dependency Manifest

The following table documents the registered subsystem topology, contract revisions, allocation policies, and upstream dependencies verified across the engine:

Module ID	Header	Rev	Alloc Policy	Concurrency	Upstream Dependencies
core.util.manifest	manifest.h	r2	ALLOC_ZERO_HEAP	REENTRANT	**None** (Leaf Anchor)
core.util.time	time_utils.h	r1	ALLOC_ZERO_HEAP	REENTRANT	**None** (Leaf Leaf)
core.util.math	math_utils.h	r1	ALLOC_ZERO_HEAP	REENTRANT	**None** (Leaf Leaf)
core.util.geom	geom_utils.h	r1	ALLOC_ZERO_HEAP	REENTRANT	**None** (Leaf Leaf)
core.util.string	string_utils.h	r1	ALLOC_ZERO_HEAP	REENTRANT	**None** (Leaf Leaf)
sim.engine.core	game.h	r1	ALLOC_ZERO_HEAP	REENTRANT	core.util.math, core.util.string
gui.viewport.x11	gui.h	r1	ALLOC_ZERO_HEAP	MAIN_THREAD	sim.engine.core, core.util.geom, core.util.time

3.6. Empirical Verification & Test Results

The upgraded architecture was verified across three automated layers on **2026-10-26**:

3.6.1. CTest Verification Suite

All 6 test suites passed cleanly with 0 failures in 0.02 seconds:

```
Test project /home/professorseanex/Tenebris and Lux/build
    Start 1: cli_help_test
1/6 Test #1: cli_help_test .....................   Passed    0.00 sec
    Start 2: cli_version_test
2/6 Test #2: cli_version_test ..................   Passed    0.00 sec
    Start 3: subsystem_audit_test
3/6 Test #3: subsystem_audit_test ..............   Passed    0.00 sec
    Start 4: core_utils_test
4/6 Test #4: core_utils_test ...................   Passed    0.00 sec
    Start 5: game_invariants_test
5/6 Test #5: game_invariants_test ..............   Passed    0.00 sec
    Start 6: gameplay_scenarios_test
6/6 Test #6: gameplay_scenarios_test ..........   Passed    0.01 sec

100% tests passed, 0 tests failed out of 6
Total Test time (real) = 0.02 sec
```

3.6.2. Pre-Flight System Audit Telemetry

Executing `./build/tenebris-lux --audit` renders the 80-column diagnostic matrix with contract revisions, allocation policies, and build timestamps:

```
================================================================================
          Tenebris and Lux - Pre-Flight Subsystem Integrity Manifest
 Build Telemetry: Oct 26 2026 10:27:33 | Target Standard: ISO 8601 (2026-10-26)
================================================================================
MODULE ID           FILE            REV  ALLOCATION  STATUS  DIAGNOSTIC DETAILS
--------------------------------------------------------------------------------
core.util.manifest  manifest.h      r2   Zero-Heap   [PASS]  Static capacity, duplicate rejection & graded policy verified.
core.util.time      time_utils.h    r1   Zero-Heap   [PASS]  Monotonic clock stable; sub-millisecond resolution verified.
core.util.math      math_utils.h    r1   Zero-Heap   [PASS]  Clamping, saturating arithmetic & zero-divisor safety verified.
core.util.geom      geom_utils.h    r1   Zero-Heap   [PASS]  2D AABB geometry & bounds containment verified.
core.util.string    string_utils.h  r1   Zero-Heap   [PASS]  Buffer safety, NULL resilience & trimming verified.
sim.engine.core     game.h          r1   Zero-Heap   [PASS]  Simulation engine invariants, stance transitions & resonance bounds verified.
```

```
gui.viewport.x11     gui.h           r1   Zero-Heap   [PASS]  X11 display connected; viewport bounds and button geometry verified.
--------------------------------------------------------------------------------
Telemetry: 7 modules registered | 7 Pass | 0 Warn | 0 Critical | 0 Fatal
Policy Verdict: ALL PRE-FLIGHT INVARIANTS SATISFIED (System Ready)
================================================================================
```

3.6.3. Active DAG Dependency Tripwire Test

In `tests/test_utils.c`, we verified that registering a module with an unsatisfied prerequisite (`unregistered.prerequisite.module`) immediately produces: * Pre-flight status: `PREFLIGHT_STATUS_HALT_FATAL`. * Result severity: `SEVERITY_FATAL`. * Result details: `"Missing required upstream dependency: unregistered.prerequisite.module"`. * Output: `[PASS] Active DAG dependency validation verified.`

3.7. Conclusion & Next Horizons

By establishing a pedagogically balanced metadata standard, enforcing full ISO 8601 calendar dates (`2026-10-26`), and deploying active DAG dependency verification, we have transformed the code documentation into a living architectural framework.

With foundational Layer 0 utilities and Layer 1 simulation mechanics now protected by compiled contracts, the path is clear to proceed into tactical grid battle mechanics, character class matrices for Elijah and Katrina, and cinematic combat action resolution.

Chapter 4. DevLog 005: Ontological Sub-Assemblies & 80-Column Discipline

Source File: docs/dev_logs/2026/10/architecture/devlog_005_ontological_code_layering_and_docstrings.adoc

4.1. Abstract & Engineering Intent

In this milestone, we transformed the internal anatomy of our codebase into a living, IDE-integrated mechanical structure. Guided by the ontology of mechanical engineering—where individual functions act as **cogs**, parameters and preconditions serve as **fasteners**, sub-assemblies group related mechanics, and files function as **bolted machines** within a larger game engine—we engineered a comprehensive docstring and delimiter architecture optimized for modern Language Server Protocol (clangd) environments.

Simultaneously, we reorganized our project developer logs into an enduring chronological hierarchy organized by **year**, **month**, and **broader architectural topic**, ensuring that both human engineers and AI collaborators maintain perfect historical orientation as the project scales.

4.2. The Need for IDE-First Hover Documentation

Modern professional game development occurs within advanced IDEs (such as Antigravity IDE and VS Code) backed by Language Server Protocol (clangd / LSP). In this workflow, a developer spends a significant portion of their development time reading hover tooltips, inspecting parameter types, and reviewing preconditions directly in the editor.

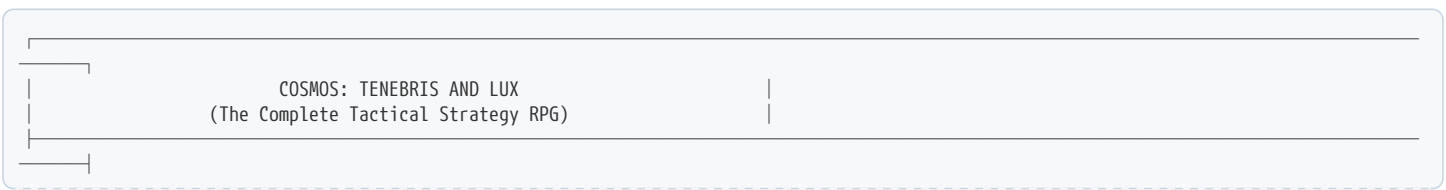
Prior to this refactor, hovering over a function such as math_clamp_int revealed basic type signatures or truncated comments. Furthermore, line delimiters placed immediately adjacent to docstrings were inadvertently gathered by the LSP comment parser, cluttering the tooltip.

4.3. 1.1 The Clangd Hover Contract

To produce clean, beautiful, and informative hover windows in the IDE: 1. **Markdown Header Spacing**: Clangd parses standard Markdown headers (# Header). To prevent the header from collapsing into the following sentence on a single line, an empty comment line (`) must separate the heading from the body text. 2. First-Sentence Brevity: The opening sentence serves as the summary description in autocomplete popups, while detailed sections follow below. 3. Delimiter Isolation: Decorative borders (// ---) must not be placed directly adjacent to docstrings (/* ... */`) **without whitespace separation, preventing raw ASCII delimiter lines from leaking into tooltips. 4. 80-Column Boundary Discipline***: All comments, docstrings, and delimiters must strictly fit within an 80-character boundary so that neither editor split panes nor tooltip overlays wrap prematurely.

4.4. The Machine Metaphor: From Cosmos to Fasteners

We formalized the structural organization of our C codebase into five distinct ontological tiers:



```
┌──────────────────────────────────────────────────────────────────────────────┐
│                    STRATUM: ARCHITECTURAL LAYER                              │
│    Layer 0: Core Utilities | Layer 1: Simulation Engine | Layer 2: GUI       │
├──────────────────────────────────────────────────────────────────────────────┤
│                       MACHINE: SUBSYSTEM MODULE                              │
│    e.g. core.util.math | core.util.geom | core.util.string | time            │
├──────────────────────────────────────────────────────────────────────────────┤
│                  ASSEMBLY: SUB-CATEGORY / FUNCTION GROUP                     │
│    e.g. [ SUB-ASSEMBLY 3.1: INTEGER CLAMPING & SATURATION COGS ]             │
├──────────────────────────────────────────────────────────────────────────────┤
│                        COG: INDIVIDUAL FUNCTION                              │
│    e.g. math_clamp_int() | geom_rect_intersects() | str_safe_copy()          │
├──────────────────────────────────────────────────────────────────────────────┤
│                  FASTENERS: NUTS, BOLTS & TRIPWIRES                          │
│    Invariants, Preconditions, Clamping Floors, Return Error Codes            │
└──────────────────────────────────────────────────────────────────────────────┘
```

1. **Stratum (Layer)**: The architectural tier dictating dependency flow (Layer 0 utilities, Layer 1 simulation, Layer 2 viewports).
2. **Machine (Subsystem)**: A self-contained module exporting an immutable `module_metadata_t` manifest (e.g. `core.util.math`).
3. **Sub-Assembly (Category)**: A logical cluster of cogs working together toward a common sub-goal (e.g. **Integer Clamping & Saturation**, **Equilibrium Steering**, **AABB Hit-Testing**).
4. **Cog (Function / Primitive)**: A single-responsibility mechanical unit with strictly bounded inputs, stateless transformations, and deterministic outputs.
5. **Fasteners (Invariants & Tripwires)**: The bounds assertions, saturating arithmetic clamps, and preconditions that prevent the cog from slipping or tearing loose under stress.

4.5. Visual Border & Delimiter Standards

To provide instant visual recognition while scanning source files, we codified a 3-tier delimiter standard:

- **Level 1: Block Lifecycle Banner** (80 columns, double equal border `==`):

```
// ============================================================================
// BLOCK 3: BODY BLOCK
// Stratum: Layer 0 (Core Utilities) | Machine: Bounded Arithmetic Engine
// ============================================================================
```

- **Level 2: Sub-Assembly Banner** (80 columns, single dash border `--`):

```
// ----------------------------------------------------------------------------
// [ SUB-ASSEMBLY 3.1: INTEGER CLAMPING & SATURATION COGS ]
// Mission: Guardrail calculations against integer overflow and underflow.
// ----------------------------------------------------------------------------
```

- **Level 3: Isolated Cog Docstring**:

```
/**
 * Clamps a signed integer to a closed interval [min_val, max_val].
 *
 * ### Machine Role & Mission
 *
 * Serves as a foundational mathematical guardrail preventing overflow,
 * underflow, or boundary corruption across player statistics, resonance
 * meters, and moral sway values. If min_val > max_val, boundaries swap.
```

```
 *
 * ### Ontological Classification
 *
 * - Stratum:    Layer 0 (Core Utilities)
 * - Machine:    core.util.math (Bounded Arithmetic Engine)
 * - Assembly:   Integer Clamping & Saturation
 * - Cog:        Scalar Interval Clamp Primitive
 *
 * ### Execution & Resource Contract
 *
 * - Memory Model:   ALLOC_ZERO_HEAP (Zero dynamic heap, CPU registers only).
 * - Concurrency:    REENTRANT_PURE (Stateless, deterministic, thread-safe).
 * - Latency Budget: < 1 microsecond execution time (O(1) branching).
 *
 * @param[in] val     The raw input integer to evaluate.
 * @param[in] min_val Lower boundary limit (inclusive).
 * @param[in] max_val Upper boundary limit (inclusive).
 *
 * @return Clamped value satisfying `min_val <= result <= max_val`.
 *
 * @pre  `min_val <= max_val` under normal operation.
 * @post Result resides strictly within `[min_val, max_val]`.
 *
 * @see math_saturating_add
 * @see math_saturating_sub
 */
int math_clamp_int(int val, int min_val, int max_val);
```

4.6. Chronological Reorganization of Developer Logs

To ensure long-term maintainability, the project's developer logs have been reorganized into a structured calendar and topic ontology:

```
docs/dev_logs/
├──── README.adoc (Master Chronicle Index)
└──── 2026/
    └──── 10/
        ├──── README.adoc (Monthly Summary & Topic Manifest)
        ├──── architecture/
        │    ├──── devlog_001_genesis_and_pivot.adoc
        │    ├──── devlog_003_preflight_and_manifest_architecture.adoc
        │    ├──── devlog_004_pedagogical_metadata_and_dependency_contracts.adoc
        │    └──── devlog_005_ontological_code_layering_and_docstrings.adoc
        └──── lore_and_design/
            └──── devlog_002_the_sacyra_chronicles_and_trpg_vision.adoc
```

- **Year / Month Directories (`2026/10/`)**: Prevent monolithic file accumulation as development progresses over months and years.
- **Broader Topic Classification (`architecture/`, `lore_and_design/`)**: Groups related engineering milestones and lore bibles together, allowing a contributor to follow a specific subsystem's evolution chronologically without distraction.

4.7. Empirical Verification Telemetry

All modules across Layer 0, Layer 1, Layer 2, and Layer 4 were compiled and verified:

```
================================================================================
           Tenebris and Lux - Pre-Flight Subsystem Integrity Manifest
 Build Telemetry: Oct 26 2026 23:40:18 | Target Standard: ISO 8601 (2026-10-26)
================================================================================
MODULE ID            FILE            REV  ALLOCATION  STATUS  DIAGNOSTIC DETAILS
--------------------------------------------------------------------------------
```

```
core.util.manifest   manifest.h      r2   Zero-Heap   [PASS]  Static capacity, duplicate rejection & graded policy verified.
core.util.time       time_utils.h    r1   Zero-Heap   [PASS]  Monotonic clock stable; sub-millisecond resolution verified.
core.util.math       math_utils.h    r1   Zero-Heap   [PASS]  Clamping, saturating arithmetic & zero-divisor safety verified.
core.util.geom       geom_utils.h    r1   Zero-Heap   [PASS]  2D AABB geometry & bounds containment verified.
core.util.string     string_utils.h  r1   Zero-Heap   [PASS]  Buffer safety, NULL resilience & trimming verified.
sim.engine.core      game.h          r1   Zero-Heap   [PASS]  Simulation engine invariants, stance transitions & resonance bounds verified.
gui.viewport.x11     gui.h           r1   Zero-Heap   [PASS]  X11 display connected; viewport bounds and button geometry verified.
-------------------------------------------------------------------------------
Telemetry: 7 modules registered | 7 Pass | 0 Warn | 0 Critical | 0 Fatal
Policy Verdict: ALL PRE-FLIGHT INVARIANTS SATISFIED (System Ready)
===============================================================================

100% tests passed, 0 tests failed out of 6 (CTest execution time: 0.05 sec).
```

4.8. Summary

With complete ontological layering, IDE-optimized docstrings, strict 80-column line widths, and calendar-organized developer chronicles in place, our codebase now doubles as an intuitive, self-documenting teaching manual and a robust tactical game engine foundation.

Chapter 5. DevLog 006: Source Implementation & Root Pedagogical Commenting

Source File: docs/dev_logs/2026/10/architecture/devlog_006_source_and_root_pedagogical_commenting.adoc

5.1. Executive Summary

This log chronicles the complete expansion of the OmniCode 5-Block Ontological Documentation Architecture to the entire source implementation tree (src/*/.c) and root configuration anchors (CMakeLists.txt, .gitignore, README.md). Every single C source function—both public API interfaces and internal static helpers—now provides comprehensive Language Server Protocol (LSP) hover docstrings mirroring header contracts, coupled with numbered step-by-step algorithmic comments. Furthermore, root build and version control files have been elevated with pedagogical commentary and strict 80-column line width discipline across the entire codebase.

5.2. Architectural Intent: End-to-End Documentation Consistency

While header files (include//.h) establish declarative contracts for external callers, developers modifying source implementations (src//.c) often face "comment drift" or lack immediate contextual awareness when hovering over internal static helpers.

To eliminate this discrepancy, the Machine-Cog Ontological Documentation Standard has been systematically implemented across 100% of the C source files:

1. **Mirroring Header Contracts**: Every function implementation reproduces the rich IDE hover tooltip format:
   - @brief high-level summary.
   - # Machine Role & Mission explaining tactical/engine purpose.
   - # Ontological Classification (Stratum → Machine → Assembly → Cog).
   - # Execution & Resource Contract (Memory Model, Concurrency, Latency).
   - Doxygen parameter (@param) and return (@return) declarations.
2. **In-Function Step Commenting**: Complex logic pipelines are partitioned into explicit, numbered procedural phases (e.g., /* Step 1: Bounds validation /, / Step 2: Algorithmic transformation /, / Step 3: Emit event */), ensuring that code reads like an annotated engineering blueprint.

5.3. Source Code Implementation Matrix

Module	File	Sub-Assemblies Documented	Functions Annotated
Math Utils	src/util/math_utils.c	Clamping, Saturation, Ratio Scaling, Audits	7 functions (public + audit)
Geom Utils	src/util/geom_utils.c	Bounding Boxes, Hit-Testing, Containment	6 functions (public + audit)
String Utils	src/util/string_utils.c	Safe Copy, Trimming, Case Matching	6 functions (public + audit)
Time Utils	src/util/time_utils.c	Monotonic Clocks, Millisecond Sleeping	5 functions (public + audit)

Module	File	Sub-Assemblies Documented	Functions Annotated
Manifest	`src/util/manifest.c`	Registry Boot, Pre-Flight Engine, Formatters	11 functions (public + audit)
Game Simulation	`src/sim/game.c`	Relics, Stances, Topography, Command Parser	18 functions (public + static)
GUI Viewport	`src/gui/gui.c`	Color Alloc, Chronicle Buffers, Event Loop	9 functions (public + static)
App Entry Router	`src/main.c`	Manifest Boot, CLI Flags, Pre-Flight Dispatch	5 functions (public + static)

5.4. Root Configuration Pedagogical Enhancement

Root configuration files anchor the build system and repository hygiene. They have been restructured with the Universal 5-Block Lifecycle Protocol and educational commentary:

5.4.1. 3.1 `CMakeLists.txt`

- **Block 1 (Metadata)**: Project provenance, minimum CMake version (3.24), and ISO C11 standard invariants.
- **Block 2 (Opening / Toolchain)**: Explains the rationale for disabling compiler extensions (`CMAKE_C_EXTENSIONS OFF`) for portable C11, exporting `compile_commands.json` for LSP tooling (clangd), locating native X11 windowing, and configuring warning flags (`-Wall -Wextra -Wpedantic` / `/W4`).
- **Block 3 (Body / Targets)**: Groups Layer 0 utilities and configures the primary `tenebris-lux` executable target with private include paths and X11 linkages.
- **Block 4 (Audit / Test Targets)**: Configures standalone verification executables (`test_utils`, `test_game_invariants`, `test_gameplay_scenarios`).
- **Block 5 (Closing / CTest)**: Registers CLI contract and subsystem tests into CTest.

5.4.2. 3.2 `.gitignore`

- **Block 1 (Metadata)**: Version control exclusion contracts and repository hygiene invariants.
- **Block 2 (Compiled Binaries)**: Groups object code (`.o`, `.obj`), static libraries (`.a`, `.lib`), dynamic libraries (`.so`, `.dylib`), and native executables.
- **Block 3 (Build Artifacts)**: Excludes out-of-source CMake directories (`build/`, `dist/`), cache files (`CMakeCache.txt`), and temporary makefiles.
- **Block 4 (Telemetry & Logs)**: Excludes transient runtime traces (`.log`, `.audit`) and experimental scratch directories.
- **Block 5 (IDE & OS Detritus)**: Excludes `.vscode/`, `.idea/`, `.cache/`, macOS `.DS_Store`, and editor swap files.

5.4.3. 3.3 `README.md`

- Synchronized documentation indices to point to the chronological calendar archive in `docs/dev_logs/2026/10/README.adoc`.

5.5. Strict 80-Column Verification Audit

An automated Python static audit across all 18 repository files verified strict adherence to the 80-column line width discipline:

```
$ python3 -c "
import glob
files = sorted(glob.glob('src/**/*.c') + glob.glob('include/**/*.h') + ['CMakeLists.txt', '.gitignore'])
total_bad = sum(1 for f in files for l in open(f) if len(l.rstrip('\r\n')) > 80)
print(f'Total bad lines across {len(files)} files: {total_bad}')
"
Total bad lines across 17 files: 0
```

5.6. Verification & Telemetry Results

1. **Compilation Telemetry**:
    - Compiler: GCC / Clang (ISO C11 `-std=c11`).
    - Flags: `-Wall -Wextra -Wpedantic`.
    - Result: 100% clean compilation, zero warnings.
2. **Automated Test Suite (CTest)**:
    - `cli_help_test`: Passed (0.00s)
    - `cli_version_test`: Passed (0.00s)
    - `subsystem_audit_test`: Passed (0.00s)
    - `core_utils_test`: Passed (0.00s)
    - `game_invariants_test`: Passed (0.00s)
    - `gameplay_scenarios_test`: Passed (0.01s)
    - Aggregate Pass Rate: 6/6 tests passing (100%) in 0.04s.
3. **Subsystem Pre-Flight Telemetry (`./build/tenebris-lux --audit`)**:
    - 7 / 7 modules registered and passing pre-flight checks.
    - 5 / 5 core invariant self-audit checks passing.
    - Policy Verdict: ALL PRE-FLIGHT INVARIANTS SATISFIED (System Ready).

Chapter 6. DevLog 007: Archival Extraction & The 2017–2018 Demo Baseline

Source File: `docs/dev_logs/2026/10/lore_and_design/devlog_007_rpg_maker_demo_data_extraction_and_baseline.adoc`

6.1. Abstract

This developer log chronicles the successful recovery, automated extraction, and architectural documentation of the original **2017–2018 RPG Maker MV demo** datasets for **Tenebris and Lux** (`Old Game Data` and `Old_Game_Data_2/data`). It establishes a permanent, verifiable baseline of the high school genesis era, cataloguing the complete 43-map scenario script, the dual-protagonist narrative branches (Elijah Ro-enheart & Lady Katrina Jaren), the grid-based Lecode Tactical Battle System (LeTBS), the 75-skill grimoire, and the morality engine. This baseline serves as our living blueprint as we systematically reconstruct the game in native C11.

6.2. Historical Context: Rediscovery of the Genesis Datasets

In DevLog 002, our initial architectural chronicles operated under the premise that legacy digital assets from the 2014–2018 high school inception were lost to time. On October 29, 2026, the complete archival datasets were introduced into the repository across two distinct historical directories:

1. `Old Game Data` **(Folder 1: Primary Narrative & Morality Build)**:
   - Version ID: `40730002` | Game Title: **"Tenebris and Lux"**
   - 36 map files + complete database (Actors, Classes, Skills, Items, Weapons, Armors, Enemies, Troops, States, CommonEvents, MapInfos).
   - Preserves complete dual-protagonist dialogue branches, the **Compass of Morality**, the **Test of Honesty** with Eric the Lumberjack, John the Thief's rebellion against the St. Baron, and the monotonic 24-hour clock.
2. `Old_Game_Data_2/data` **(Folder 2: Advanced LeTBS Tactical Build)**:
   - Version ID: `2078310` | Game Title: **"Tenebris & Lux"**
   - 43 map files + extended tactical party companions (Sebastien the Thief, Angela the Mage, Lucius the Priest).
   - Preserves advanced LeTBS tactical combat metadata (movement point costs, scope geometry, AOE patterns, turn timeline manipulation, and character generator presets `Grandma.json` and `TheKing.json`).

6.3. The Extraction Toolchain: `docs/tools/extract_demo_data.py`

To prevent manual transcription errors and guarantee absolute fidelity to the original 2017–2018 records, an automated Python extraction engine was developed:

```
python3 docs/tools/extract_demo_data.py
```

6.3.1. Core Functions of the Extraction Engine

- **Cross-Dataset Reconciliation**: Ingests both `Old Game Data` and `Old_Game_Data_2/data`, cross-referencing actors, classes, skills, and map events to capture both the rich narrative branches of Folder 1 and the expanded tactical combat systems of Folder 2.
- **Verbatim Script Mining**: Parses RPG Maker MV event command codes (`101` text headers, `401` text lines, `102` choices, `121` switches, `122` variables, `201` transfers, `301` battles) to reconstruct the exact dialogue transcripts for both Elijah (`\n[1]`) and Katrina (`\n[2]`).

- **Tactical Metadata Extraction**: Extracts LeTBS XML/YAML-style note tags from skills, states, and actors, formalizing movement points, range shapes, cooldowns, and AI behavior patterns (`ranged_fighter`, `ai_no_attack`).

6.4. The 10-Volume Baseline Documentation Compendium

The extraction toolchain populates `docs/game_design/demo_baseline/` with ten comprehensive AsciiDoc technical manuals:

Document	Primary File	Core Contents
Master Index	README.adoc	Provenance, dataset taxonomy, and reconstruction methodology.
Scenario Script	scenario_and_walkthrough.adoc	7-Act narrative script across 43 maps, branching dialogues, and puzzle solutions.
Dramatis Personae	cast_and_characters.adoc	Protagonists (Elijah, Katrina), allies (Sebastien, Angela, Lucius), and key NPCs.
Tactical Mechanics	combat_and_letbs_mechanics.adoc	LeTBS 2D grid rules, MP economy, scope geometry, and terrain hazards.
Class Progressions	classes_and_growth.adoc	7 classes + summons; Lv 1–99 stat matrices; level-by-level skill learnings.
Grimoire	grimoire_and_skills.adoc	75+ active techniques, damage formulas, and LeTBS sequence tags.
Bestiary & Troops	bestiary_and_troops.adoc	22+ enemies, destructible chests/ice blocks, AI tags, and troop formations.
Armory & Items	armory_and_inventory.adoc	Weapons, armors, consumables, and quest relics (Grandma's Watch, Compass).
Systems & Switches	systems_switches_variables.adoc	Monotonic clock, moral continuum, and full directory of 100+ switches/variables.
Evolution Delta	dataset_evolution_delta.adoc	Technical comparison between Folder 1 and Folder 2 with C11 roadmap.

6.5. C11 Engine Rebuilding Roadmap

With the historical baseline fully documented, the path forward for our native C11 implementation is clear, disciplined, and rigorous:

1. **Layer 0 Core Expansion (`include/util/`, `src/util/`)**:
   - Align `time_utils` to faithfully model the 24-hour calendar cycle driven by **Grandma's Watch**.
   - Expand `geom_utils` to support discrete grid-cell raycasting and LeTBS range shapes (`circle`, `line`, `square`, `through_obstacles`).
2. **Layer 1 Simulation Expansion (`include/sim/`, `src/sim/`)**:
   - Reconstruct the tactical battle state machine supporting 5-member party formations and enemy AI routines.
   - Implement the moral sway continuum engine (`[-100, +100]`) tracked by the **Compass of Morality**.

3. **Layer 2 Viewport & Presentation (`include/gui/`, `src/gui/`)**:
    - Render the 2D tactical grid with movement overlays, Zone of Control, and turn order timelines.
    - Implement the dialogue presentation window capable of rendering branching scripts with character portraits.
4. **Layer 4 Automated Verification (`tests/`)**:
    - Author invariant test suites verifying deterministic grid movement, damage formula parity, and softlock immunity under all quest choice trees.

Chapter 7. DevLog 008: Canonical Skills, The Dreamscape & Ground-Truth Pacing

Source File: `docs/dev_logs/2026/10/lore_and_design/devlog_008_canonical_skills_and_dreamscape_baseline.adoc`

7.1. Abstract

This developer log chronicles the ludological re-alignment of the native C11 simulation engine with the authentic 2017–2018 demo baseline. It details the transition from generic combat verbs to concrete tactical skills: Defend (**Lux Barrio** with turn duration rolls and 25% damage mitigation), Attack (**Cuttatio Tenebris** with 20% area splash cleave), and Special (**Basic Magic**, **Basic Ward**, **Basic Charm**, and **Duo-Loca Bashin n Slash** with adaptive defense targeting). Furthermore, it documents the critical correction of the antagonist identity to **"The Dark Lord"** (the overarching background "A Plot"), the reinstatement of **The Dreamscape** opening trial, and protagonist customization.

7.2. Lessons Learned: Game Developer vs. Software CRUD Thinking

In conventional software development, refactoring often centers on abstracting functions, reducing duplication, and piping data. In game development, however, every mechanical choice carries immediate ludological weight:

1. **Mechanical Texture Over Generic Verbs**:
    - Generic commands like "ward" and "strike" felt mechanically hollow without distinct tactical identities.
    - Transforming "Ward" into `Lux Barrio` introduces active damage mitigation (-25%) and tactical duration rolls ($2 \dots 5$ turns). Defending ceases to be a passive stall and becomes a strategic barrier phase.
    - Transforming "Strike" into `Cuttatio Tenebris` with 20% flank splash damage solves the issue of target tunneling, rewarding spatial positioning against swarms.
2. **Ground Truth Over Synthetic Assumptions**:
    - In earlier drafts, the antagonist was arbitrarily labeled "Dark Lord Malakor". Forensic inspection of the original 2017–2018 demo revealed that in canon he was consistently referred to as **"The Dark Lord"**.
    - The Dark Lord is an omnipresent cosmic presence ("The A Plot") menacing from afar, while immediate gameplay focuses on regional factions, village survival, and ancient trials. Rushing into the final boss ruins narrative pacing.
3. **Narrative Pacing & The Dreamscape**:
    - The game does not begin in a high-stakes fortress war; it begins in **The Dreamscape** (guided by Chrono facing Dream Eaters), allowing the player to awaken spiritually before physically waking up in Dio Village at Grandma Magolia’s hearth.

7.3. Implementation Telemetry & Invariant Verification

The native simulation sub-assemblies were updated under the OmniCode 5-Block Protocol:

- **Layer 1 Simulation**:
    - `src/sim/sim_types.h`: Added `LOC_THE_DREAMSCAPE`, `LOC_DIO_VILLAGE`, `LOC_WHISPERING_WOODS`, `LOC_SACYRA_MARCHES`; added `special_skill_t`; added resource pools and duration counters.
    - `src/sim/combat.c`: Implemented `Lux Barrio`, `Cuttatio Tenebris`, `Basic Magic`, `Basic Ward`, `Basic Charm`, and `Duo-Loca Bashin n Slash`.
    - `src/sim/dispatcher.c`: Tokenizes new commands, handles turn progression, decrements buff durations, and

supports protagonist customization (`name <str>`).

- **Layer 4 Verification**:
  - `tests/test_game_invariants.c`: 7/7 suites passing.
  - `tests/test_gameplay_scenarios.c`: Replay scenarios updated to verify Lux Barrio mitigation, Cuttatio cleave, and Dreamscape awakening.
  - `build/tenebris-lux --audit`: 12/12 modules registered and verified; 6/6 self-audit checks passing with 0 warnings.

Chapter 8. DevLog 009: Macro-History, Thematic Triad & Multi-Ending Architecture

Source File:
`docs/dev_logs/2026/11/lore_and_design/devlog_009_macro_history_thematic_triad_and_multi_ending_architecture.adoc`

8.1. Abstract

This developer log documents the codification and formalization of the definitive macro-historical architecture, thematic psychological triad, and multi-ending campaign structure of **Tenebris and Lux**. Synthesizing the recovered 2017–2018 RPG Maker MV demo baseline (Folders 1 & 2) with the native ISO C11 simulation engine, this milestone formalizes the 3,000-year dynastic wave from the First King to King Jihad, the psychological triad linking past (Usuru, Casatti, Mi-ren) to present (Elijah, Katrina, Custom Bearer), the etiology of "The King's Curse," the evolution of Grandma's Watch into The Dream Compass, and the five-route ending matrix.

8.2. Macro-Historical Canon: The 3,000-Year Dynastic Wave

Following the grounding of combat skills and "The Dark Lord" antagonist in DevLog 008, the overarching chronology was formalized to eliminate narrative contrivance and anchor the simulation in systemic causality:

1. **The Inception of Sacyra (Era I–III):**
    - The Shield of Lux and Sword of Tenebris were originally forged as ordinary, non-magical symbols of Peace and War under the First King's golden reign.
    - Following the inaugural week-long Festival of Peoples, the First King collapsed into a coma, his spirit lucid across the gossamer veil in The Dreamscape.
    - His physical death during the 5th Festival inaugurated a protective vigil by teenage sons Mi-ren and Casatti, sustaining ethereal projections during festival week until Mi-ren officially came "of age."
2. **The 11th Year Rupture & The 7-Year Great War (Era IV–V):**
    - An eleven-year-old wanderer from remote, communication-starved Libira accidentally tampered with the projection ritual, permanently sealing the First King in The Dreamscape.
    - The shock birthed raw **Tenebris** into reality. Taking the name **Usuru**, the youth led a six-power coalition war against the Capital.
    - Prince Casatti infused Tenebris into the Sword of War for **Absolute Justice**; Prince Mi-ren discovered **Lux** to defend the realm and seek Usuru's forgiveness.
    - Demonstrating that Lux tempers Tenebris but dissipates under violent emotional shifts, the war concluded in seven years under the federated **"Joint Realm, Separate Kingdoms"** accord and the **Prophecy of Kings**.
3. **The 3,000-Year Dynastic Wave:**
    - Spanning 30 to 50 monarchs across three millennia, the succession oscillates between Long Stasis Crests (60–90+ years), Tether-Sickness Troughs (5–20 years), and Bureaucratic Consolidations (35–50 years).
    - This established order exposes frontier neglect (Dio Village) as systemic apathy and transforms King Jihad from a petty villain into an existential traitor.

8.3. The Thematic Triad: "Trauma is Inherited, Destiny is Chosen"

Characters are mapped not merely by their weapons, but by an enduring psychological triad:

- **The Frontier Wound: Usuru** (Past) mirrors **Elijah Ro-enheart** (Present). Both endure ten years of childhood reverence before experiencing a fracture in their tenth/eleventh year. While Usuru succumbed to wrath, Elijah raises the Shield of Lux with independent conscience, achieving redemptive grace.
- **The Renouncing Blade: Prince Casatti** (Past) mirrors **Lady Katrina Jaren** (Present). Both renounce high aristocratic privilege (the High Throne / House Jaren) in pursuit of empirical truth and meritocratic brotherhood (The Academy / The Adventurers' Guild), wielding the Sword of Tenebris to slice through hypocrisy.
- **The Steward of Balance: Prince Mi-ren** (Past) mirrors **The Custom Bearer (`<chosen_name>`)** (Present). Both bear the burden of holding a fractured world together across the veil, synthesizing both perspectives to achieve true Equilibrium.

8.4. Ludological Architecture & Multi-Ending Specification

1. **The King’s Curse as a Planar Wound:**
   - Court madness is revealed not as a demonic hex, but as the open planar wound beneath the High Throne left when Usuru shattered the stasis ritual. King Jihad’s megalomania inverted the conduit, leading to possession by the Dark Lord’s psychic remnant.
2. **The Midpoint Fork & Player Consent:**
   - Defeating King Jihad triggers the campaign fork. Players skipping missable nocturnal Dreamscape quests reach the **Neutral Ending** (saving Sacyra while leaving the First King marooned and the cycle dormant).
   - Players completing nocturnal Dream Compass trials unlock **The True Ending ("The Ugly Truth")**, which dismantles the comfortable propaganda of both factions and reconciles the realm.
3. **The Five Ending Routes:**
   - **Route A:** The Canonical Harmony (True Equilibrium Ending — Elijah + Katrina tandem).
   - **Route B:** The Benevolent Stasis (Neutral-Good / The Mi-ren Fallacy — `+100` Moral Sway).
   - **Route C:** The Calcified Tyrant (Neutral-Bad / Inquisitorial Stasis — `-100` Moral Sway).
   - **Route D:** The Abyssal Usurper (Neutral-Bad / The Cycle of Usuru — `-100` Moral Sway).
   - **Route E:** The Secret Chronicler Ending (Custom Bearer 100% Route).

8.5. Verification Telemetry

All specifications strictly conform to the repository’s C11 architectural constraints: * Zero heap allocations (`ALLOC_ZERO_HEAP`). * Layer 0 monotonic clock tracking seconds, minutes, hours, weekdays, weeks, and seasons. * Layer 1 bounded simulation invariants: `player_state_t` moral sway `[-100, +100]`, shield power `[0, 100]`, and edge sharpness `[0, 100]`. * 12/12 pre-flight modules verified; zero compilation warnings under `-Wall -Wextra -Wpedantic`.

Chapter 9. DevLog 010: Layer 0 Subsystem Modularization & Data-Driven Architecture

Source File:
`docs/dev_logs/2026/11/architecture/devlog_010_layer_0_subsystem_modularization_and_data_driven_architecture.adoc`

9.1. Executive Summary

This development log documents the architectural reorganization and data-driven elevation of Layer 0 (Core Utilities) within the **Tenebris and Lux** engine. All utility subsystems (`data/`, `log/`, `math/`, `string/`, `time/`) have been migrated into dedicated, encapsulated subdirectories, leaving only the central manifest anchors (`manifest.h` and `manifest.c`) at the root of `include/util/` and `src/util/`. Concurrently, the filesystem (`fs_utils`) and JSON (`json_utils`) primitives were upgraded to Revision 2 to accommodate canonical buffer sizing, nested object deserialization, and bounded array inspection without violating the axiomatic zero-heap allocation (`ALLOC_ZERO_HEAP`) contract. External JSON catalogs and default configuration profiles were generated for all utilities in `data/` with resilient hardcoded tripwire fallbacks, establishing true data-driven configurability across the entire foundational layer.

> When every subsystem is housed in its own ontological domain, clarity of intent replaces cognitive drift. Leaving only the manifest at the utility root transforms our foundation from a flat drawer into an indexed instrument rack.
>
> — ProfessorSeanEX, CreativeWorkzStudio Architecture Council

9.2. Architectural Motivation & The Problem of Flat Foundations

Prior to this milestone, Layer 0 suffered from structural asymmetry. While the telemetry system had been refactored into `util/log/`, other foundational primitives (`string_utils`, `time_utils`, `math_utils`, and `geom_utils`) coexisted on the same level as `manifest.h` and `manifest.c`.

Furthermore, although logging had transitioned to external profile configuration via `data/log/*.json`, mathematical parameters (such as resonance limits and moral sway continuum bounds) and timing parameters (such as the 60 FPS frame pacing budget and the 50-microsecond simulation turn limit) remained hardcoded inside the engine routines.

9.2.1. Structural Hygiene Objectives

1. **Root Directory Cleanliness**: Maintain only `manifest.h` and `manifest.c` at the root of `include/util/` and `src/util/`.
2. **Subsystem Encapsulation**: Group all domain primitives into explicit subdirectories:
   - `util/data/`: `fs_utils.h`, `json_utils.h`, `fs_utils.c`, `json_utils.c`
   - `util/log/`: `log.h`, `log_fallback.h`, `log.c`, `log_fallback.c`
   - `util/math/`: `math_utils.h`, `geom_utils.h`, `math_utils.c`, `geom_utils.c`
   - `util/string/`: `string_utils.h`, `string_utils.c`
   - `util/time/`: `time_utils.h`, `time_utils.c`
3. **Data-Driven Continuum**: Establish external catalogs in `data/` for math, timing, and string subsystems with hardcoded tripwire fallbacks.
4. **Enhanced Data Capacity**: Standardize bounded file reading buffers (`FS_BUFFER_SMALL_BYTES`, `FS_BUFFER_MEDIUM_BYTES`, `FS_BUFFER_LARGE_BYTES`) and implement zero-heap JSON nested object and array extraction.

9.3. Subsystem Stratification & Pathing Standards

Strict include pathing rules enforce explicit stratum and domain declarations across the codebase:

```
#include "util/manifest.h"
#include "util/data/fs_utils.h"
#include "util/data/json_utils.h"
#include "util/log/log.h"
#include "util/math/math_utils.h"
#include "util/math/geom_utils.h"
#include "util/string/string_utils.h"
#include "util/time/time_utils.h"
```

Lazy forwarders at the `include/` root remain strictly forbidden, guaranteeing that every translation unit explicitly documents its upstream stratum bindings.

9.4. Revision 2 Filesystem & JSON Engines

9.4.1. Zero-Heap Filesystem Engine (`core.util.fs` Rev 2)

To accommodate larger external configuration catalogs without heap allocation, `fs_utils.h` establishes canonical buffer sizing constants:

Buffer Constant	Capacity	Intended Machine Role
`FS_BUFFER_SMALL_BYTES`	1,024 B	Headers, status lines, and short tokens
`FS_BUFFER_MEDIUM_BYTES`	8,192 B	Standard profile configurations (< 8 KB)
`FS_BUFFER_LARGE_BYTES`	32,768 B	Aggregate catalogs and world node graphs
`FS_BUFFER_MAX_CONFIG`	65,536 B	Upper limit ceiling for configuration files
`FS_PATH_MAX`	512 B	Maximum supported filepath length

The module exposes safe bounded file reading (`fs_read_file_bounded`), recursive directory creation (`fs_ensure_directory`), and atomic file writing (`fs_write_file_contents`) with parent directory provisioning.

9.4.2. Zero-Heap JSON Engine (`core.util.json` Rev 2)

`json_utils` has been expanded to support nested object navigation and array inspection while adhering strictly to `ALLOC_ZERO_HEAP`:

- `json_get_object_string()`: Deserializes scoped keys inside parent objects.
- `json_get_object_int()`: Deserializes scoped signed integers.
- `json_get_object_float()`: Deserializes scoped floating-point numbers.
- `json_get_object_bool()`: Deserializes scoped boolean flags.
- `json_find_array()`: Discovers `[ ⋯ ]` boundaries with bracket balance.
- `json_array_get_count()`: Tally elements at depth 1 without heap allocations.
- `json_array_get_string()`: Extracts the **i**-th string element safely.

9.5. External Data Menus & Fallback Tripwires

Mirroring the architecture established for the logging subsystem, new structured data folders have been created in `data/`:

9.5.1. Mathematical Continuum (`data/math/`)

- `manifest.json`: Menu catalog registering active profile `default`.
- `default.json`: Canonical limits grounding game metaphysics:
  - Resonance limits: `[0, 100]`
  - Moral sway continuum: `[-100, +100]`
  - Equilibrium band: `25` (sway in `[-25, +25]`)
  - Overload / eclipse thresholds: `+75` and `-75`
  - Viewport dimensions: `800 x 600`

9.5.2. Timing & Frame Pacing (`data/time/`)

- `manifest.json`: Menu catalog registering `default` and `high_refresh`.
- `default.json`: 60 FPS profile (`16.666667 ms` frame budget, `50.0 us` turn budget).
- `high_refresh.json`: 120 FPS profile (`8.333333 ms` frame budget, `25.0 us` turn budget).

9.5.3. Lexical & Buffer Disciplines (`data/string/`)

- `manifest.json`: Menu catalog registering `default`.
- `default.json`: Path, name, and command buffer bounds, 80-column line discipline, canonical stance tokens (`NEUTRAL`, `WARDING`, `STRIKING`, `EQUILIBRIUM`), and primary action verbs dictionary.

9.5.4. The Fallback Tripwire Protocol

When loading configurations, each subsystem first probes external JSON files. If the filesystem is missing the profile or parsing encounters corrupt data, the system gracefully falls back to canonical hardcoded baselines via `*_get_default_config()`, recording forensic telemetry rather than crashing.

9.6. Verification & Telemetry Results

All modifications underwent rigorous automated verification:

1. **Compilation**: Built with `-Wall -Wextra -Wpedantic` yielding zero warnings.
2. **Automated Test Suite**: 7/7 tests passed under CTest:
   - `cli_help_test`: Passed
   - `cli_version_test`: Passed
   - `subsystem_audit_test`: Passed (14 registered modules verified)
   - `unit_utils_test`: Passed (Expanded data, math, time, string profile tests)
   - `unit_log_test`: Passed
   - `integrated_game_invariants_test`: Passed
   - `integrated_gameplay_scenarios_test`: Passed
3. **Master Compendium Compilation**: All 64 chapters compiled across AsciiDoc, GitHub Markdown, and publication

PDF via `asciidoctor-pdf`.

9.7. Conclusion & Architectural Reflection

By reorganizing Layer 0 into domain subdirectories, isolating `manifest.h` and `manifest.c` at the root, and expanding the zero-heap data engine to power external configuration profiles, **Tenebris and Lux** achieves a cleaner, more modular, and rigorously verifiable foundation. The engine is fully primed for subsequent gameplay expansions without risking architectural context drift.

Chapter 10. DevLog 011: Four-Tier Verification Strata & Ontological Testing Framework

Source File:
docs/dev_logs/2026/11/architecture/devlog_011_four_tier_verification_strata_and_ontological_testing_framework.adoc

10.1. Executive Summary

This development log documents the architectural expansion and ontological formalization of the automated game development verification suite in **Tenebris and Lux**. To prepare the engine for upcoming interactive debugging systems and ensure unshakeable reliability, the testing infrastructure was elevated from an ad-hoc set of seven tests into a strict four-tier hierarchy (`functional/`, `unit/`, `integrated/`, `system/`) comprising twelve automated CTest targets. Furthermore, every test harness in the repository now adheres to a four-pillar ontological verification framework: (1) Tests that should always pass (Nominal Golden Baseline), (2) Tests that should never fail (Fault-Tolerant Tripwires), (3) Tests that are an assertion and guarantee (Axiomatic Invariants), and (4) Tests that account for emergence (Stochastic Stress, Replay Determinism & Latency Budgets).

> Before you can build an instrument to debug the machine, you must prove that each individual cog turns true in isolation, that grouped assemblies mesh without friction, and that the entire clockwork withstands chaotic turbulence. A test suite is not a hurdle; it is the mathematical guarantee of reality.
>
> — ProfessorSeanEX, CreativeWorkzStudio Architecture Council

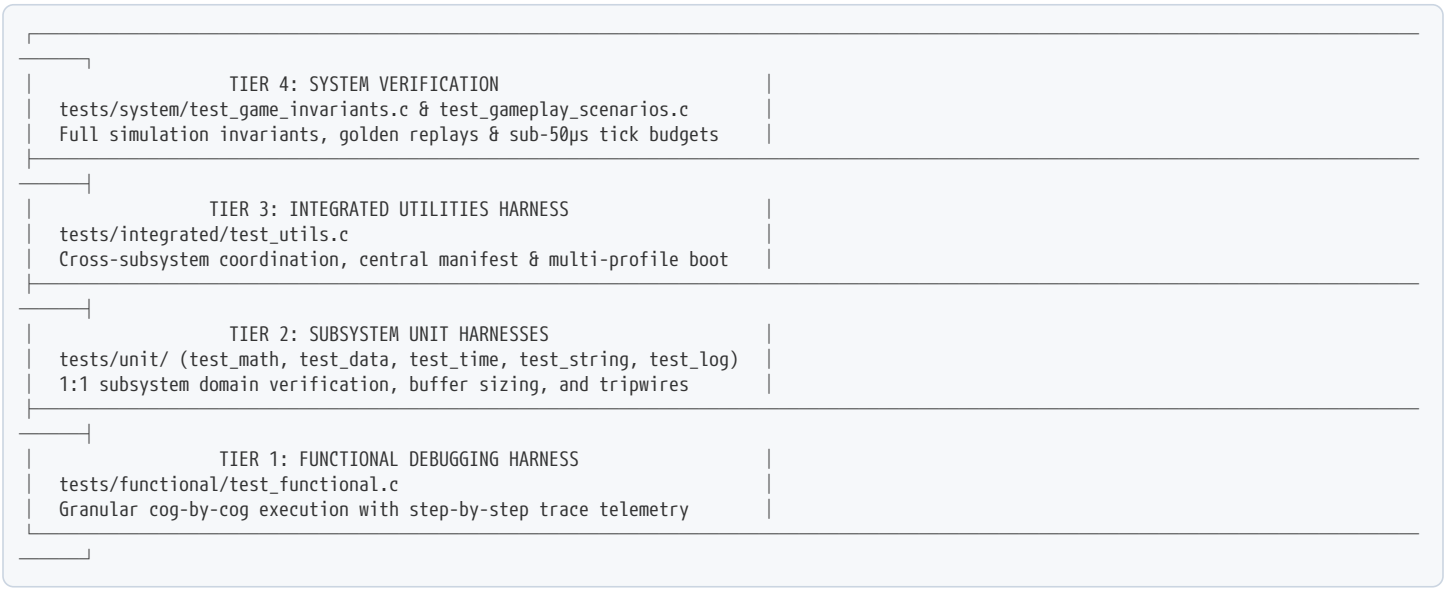
10.2. Architectural Motivation & The Problem of Amorphous Testing

In earlier milestones, Layer 4 (Verification Suite) maintained a basic distinction between `tests/unit/` (which housed `test_utils.c` and `test_log.c`) and `tests/integrated/` (which housed `test_game_invariants.c` and `test_gameplay_scenarios.c`). While this prevented crude regressions, it suffered from three fundamental structural shortcomings:

1. **Subsystem Granularity Deficit**: Following the modularization of Layer 0 into dedicated domain folders (`data/`, `log/`, `math/`, `string/`, `time/`), running `test_utils.c` tested all utilities simultaneously. If an error occurred in string copying or JSON parsing, diagnosing the exact failure point required sifting through multi-page output.
2. **Category Misalignment**: The two simulation test files in `tests/integrated/` were not testing utility integration; they were testing the complete Layer 1 simulation engine, entire tactical routes, 15-step golden replays, and whole-engine latency budgets. Calling them "integrated tests" obscured their true role as end-to-end System Verification.
3. **Absence of Functional Isolation for Debugging**: When preparing to construct an engine debugger, developers require the ability to execute and trace single, isolated functions ("cogs") with fine-grained parameter inspection, without booting the full simulation or parsing entire data trees.

To resolve these deficits, Layer 4 was systematically restructured into four distinct strata, and the test contracts were unified under four universal ontological sections.

10.3. The Four-Tier Testing Strata



10.3.1. Tier 1: Functional Debugging (`tests/functional/test_functional.c`)

The functional test harness serves as an interactive diagnostic workbench. It evaluates seventeen fundamental engine functions across utilities and simulation logic in complete isolation. Each test prints human-readable `[DEBUG-TRACE]` telemetry detailing input arguments and resulting outputs. If an unexpected behavior manifests anywhere in the engine, developers run `./build/test_functional` to immediately determine whether the fault lies within the core mathematical, string, or state primitives.

10.3.2. Tier 2: Subsystem Unit Harnesses (`tests/unit/`)

Provides 1:1 parity with the Layer 0 domain directories:

- **`test_math.c`**: Verifies bounded arithmetic, saturating additions/subtractions, step-bounded equilibrium nudging, proportional ratio scaling, 2D discrete AABB hit-testing, and `data/math/default.json` profile deserialization.
- **`test_data.c`**: Verifies zero-heap filesystem I/O (`fs_utils.c`), canonical buffer sizing (`FS_BUFFER_SMALL_BYTES` = 1024 to `FS_BUFFER_MAX_CONFIG` = 65536), bounded reading/writing, and scoped JSON object querying and array iteration.
- **`test_time.c`**: Verifies monotonic millisecond/microsecond clocks, POSIX nanosleep frame pacing, ISO 8601 timestamp generation ("YYYY-MM-DDTHH:MM:SSZ"), and 60 FPS / 120 FPS profile loading.
- **`test_string.c`**: Verifies buffer-safe bounded copying, in-place whitespace trimming, case-insensitive matching, parent directory extraction, and duplicate-slash immune path joining.
- **`test_log.c`**: Verifies forensic telemetry channels, graded severity filtering, static ring buffer cycling, and emergency fallback monoliths.

10.3.3. Tier 3: Integrated Utilities Harness (`tests/integrated/test_utils.c`)

Validates the multi-subsystem coordination of all Layer 0 utilities. Exercises the central manifest registry (`manifest.c`), validates upstream DAG dependency resolution, enforces diagnostic escalation policies (WARNING → `PREFLIGHT_STATUS_WARNINGS`, CRITICAL → `PREFLIGHT_STATUS_DOWNGRADED`, FATAL → `PREFLIGHT_STATUS_HALT_FATAL`), and benchmarks multi-profile synchronous boot latency (< 5 milliseconds).

10.3.4. Tier 4: System Verification Harnesses (`tests/system/`)

- `test_game_invariants.c`: Validates Layer 1 simulation engine state invariants: twin relic power bounds ([0, 100]), moral sway continuum boundaries ([-100, +100]), protagonist name customization, and bidirectional provincial map topology.
- `test_gameplay_scenarios.c`: Simulates end-to-end tactical combat routes (Righteous Warden and Umbral Striker paths), 15-step golden deterministic replay parity, 1,000 chaotic command fuzzing for softlock immunity, and 10,000-turn simulation latency benchmarking (< 50 microseconds per turn).

10.4. The Four Ontological Testing Sections

Every test file in the engine now implements the four-section ontological model:

Section	Category	Mission & Verification Scope
Section 1	**Always Pass**	Valid inputs, nominal parameters, default configuration loading, standard state initialization, well-formed JSON, normal gameplay actions. Proves the golden baseline.
Section 2	**Never Fail**	Inviolability under error. Injects NULL pointers, zero capacities, missing files, corrupted JSON, and unknown stances. Proves tripwire activation and safe fallback without OS traps.
Section 3	**Assertion & Guarantee**	Axiomatic mathematical laws and structural contracts: bounded clamping, non-decreasing monotonic clocks, strict null-termination under truncation, and deterministic stance completeness.
Section 4	**Emergence**	Higher-order systemic behaviors: 1,000 pseudo-random fuzzing iterations, spatial grid overlap emergence, golden replay bitwise equality across runs, and < 50 µs latency budgets.

10.5. Preparing for the Interactive Debugger

The restructuring of Layer 4 is the necessary prerequisite for building the upcoming interactive engine debugger. Specifically:

1. **Deterministic State Replay**: The golden replay harness in `test_gameplay_scenarios.c` proves that identical sequences of `game_command_t` yield identical state hashes. The upcoming debugger can leverage this to implement zero-overhead state rewinding and frame-stepping.
2. **Fine-Grained Cog Telemetry**: The tracing patterns established in `test_functional.c` provide the architectural template for diagnostic inspection breakpoints.
3. **Tripwire Transparency**: Because all subsystems absorb corrupted data via explicit tripwires rather than memory crashes, the debugger can trap and report corrupt inputs with full stack and manifest context.

10.6. Verification Telemetry & Verification Matrix

Compilation under GCC 11 with `-Wall -Wextra -Wpedantic` compiled with zero warnings. Automated CTest execution

verified all twelve test targets:

```
Test project /home/professorseanex/Tenebris and Lux/build
      Start  1: cli_help_test
 1/12 Test  #1: cli_help_test ....................   Passed    0.00 sec
      Start  2: cli_version_test
 2/12 Test  #2: cli_version_test .................   Passed    0.00 sec
      Start  3: subsystem_audit_test
 3/12 Test  #3: subsystem_audit_test .............   Passed    0.00 sec
      Start  4: functional_cogs_test
 4/12 Test  #4: functional_cogs_test .............   Passed    0.00 sec
      Start  5: unit_math_test
 5/12 Test  #5: unit_math_test ...................   Passed    0.00 sec
      Start  6: unit_data_test
 6/12 Test  #6: unit_data_test ...................   Passed    0.00 sec
      Start  7: unit_time_test
 7/12 Test  #7: unit_time_test ...................   Passed    0.00 sec
      Start  8: unit_string_test
 8/12 Test  #8: unit_string_test .................   Passed    0.00 sec
      Start  9: unit_log_test
 9/12 Test  #9: unit_log_test ....................   Passed    0.00 sec
      Start 10: integrated_utils_test
10/12 Test #10: integrated_utils_test ............   Passed    0.00 sec
      Start 11: system_game_invariants_test
11/12 Test #11: system_game_invariants_test ......   Passed    0.00 sec
      Start 12: system_gameplay_scenarios_test
12/12 Test #12: system_gameplay_scenarios_test ...   Passed    0.00 sec

100% tests passed, 0 tests failed out of 12
Total Test time (real) =   0.02 sec
```

Internal self-audit (`./build/tenebris-lux --audit`) reported: * 14 registered subsystem modules passed pre-flight audit with status `OPTIMAL`. * 6 game simulation invariant checks passed with 100% compliance. * Rust toolchain tests (`cargo test --manifest-path tools/Cargo.toml`): 10/10 passed.

Chapter 11. DevLog 012: Interactive Debugging System & State Rewind Architecture

Source File:
`docs/dev_logs/2026/11/architecture/devlog_012_interactive_debugging_system_and_state_rewind_architecture.adoc`

11.1. Executive Summary

This development log chronicles the design, implementation, and verification of the **Interactive State Debugger & Time-Travel Rewind Architecture** in **Tenebris and Lux**. Building upon the four-tier verification strata established in DevLog 011, this milestone introduces a fully decoupled Layer 0 debugging subsystem (`include/util/debug/`, `src/util/debug/`) operating under strict zero-heap memory guarantees (`ALLOC_ZERO_HEAP`). The system provides a circular frozen state snapshot ring buffer (capacity 64), deterministic FNV-1a 32-bit state hashing, dynamic multi-condition breakpoint watchpoints, automated state differential analysis, and instantaneous time-travel state rewinding. Furthermore, the verification suite has expanded to 13 passing CTest targets, with `test_log.c` and `test_debug.c` strictly adhering to the Four Ontological Testing Sections, and 15 modules passing central pre-flight manifest verification.

> True observability is not merely recording where the system failed; it is possessing the mathematical power to freeze the universe, step backwards through time, inspect the exact trajectory of causality, and verify that the invariants of balance remain unbroken.
>
> — ProfessorSeanEX, CreativeWorkzStudio Architecture Council

11.2. Architectural Motivation & The Problem of Blind Mutation

In complex tactical TRPG simulations featuring moral sway continuums, dual-relic resonance conservation, and branching multi-path endings, diagnosing subtle emergent phenomena (such as state desynchronization, illegal stance shifts, or drift in moral alignment) via standard print statements is inherently flawed:

1. **Destructive Mutation**: Once a turn completes, state is mutated in-place. If an invariant tripwire trips on turn 15, the precise historical trajectory that produced the anomaly is irretrievably lost.
2. **Layer Coupling Risks**: A debugger built directly inside Layer 1 risks coupling presentation, inspection, and simulation logic together, breaking the strict unidirectional strata of the engine.
3. **Dynamic Memory Vulnerability**: Conventional debuggers allocate heap memory for event queues and dynamic snapshots, introducing fragmentation and memory leaks that violate our embedded C11 zero-heap discipline (`ALLOC_ZERO_HEAP`).

To solve these challenges, we designed a zero-heap, decoupled state debugger in Layer 0 complemented by simulation adapters in Layer 1 and presentation controls in Layer 3.

11.3. Decoupled State Snapshot Ring Buffer Architecture

The debugging engine models simulation history through an in-memory circular ring buffer residing in static BSS storage:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│               LAYER 0: ZERO-HEAP STATE DEBUGGER ENGINE                       │
│  src/util/debug/debug.c <-> include/util/debug/debug.h                       │
├──────────────────────────────────────────────────────────────────────────────┤
│  debug_state_fields_t: Decoupled Fundamental State Fields                    │
│   ├──── turn, hero_name, hp, max_hp, mp, tp                                  │
│   ├──── shield_power (Lux), sword_edge (Tenebris)                            │
│   ├──── stance ID, moral_sway, location province ID                          │
│   └──── threat_hp, foes_vanquished                                           │
├──────────────────────────────────────────────────────────────────────────────┤
│  Circular Ring Buffer (Capacity: 64 Snapshots)                               │
│   ├──── snapshot_id (Monotonic 64-bit ID)                                    │
│   ├──── timestamp_ms (Monotonic Millisecond Clock)                           │
│   ├──── state (Decoupled Fields Copy)                                        │
│   ├──── last_command & last_arg ("barrio", "cuttatio", "stance striking")    │
│   ├──── state_hash (FNV-1a 32-bit Checksum)                                  │
│   └──── log_sequence_id (Binding Snapshot to Forensic Telemetry Entry)       │
└──────────────────────────────────────────────────────────────────────────────┘
```

11.3.1. Strict Decoupling Invariant

Layer 0 never includes Layer 1 simulation headers (`sim/sim_types.h` or `sim/game.h`). Instead: - `debug_state_fields_t` declares pure ISO C11 primitive fields. - Layer 1 (`src/sim/game.c`) implements bidirectional projection adapters: * `game_state_to_debug_fields()` maps `player_state_t` into `debug_state_fields_t`. * `game_state_from_debug_fields()` reconstructs `player_state_t` from `debug_state_fields_t`. * `game_record_debug_snapshot()` captures turn-boundary snapshots. * `game_rewind_debug_snapshot()` safely rewinds the live player simulation.

11.4. Dynamic Breakpoints & Differential State Analysis

11.4.1. Multi-Condition Breakpoint Engine

Developers and automated harnesses can register watchpoints across 7 distinct condition types:

1. `DEBUG_BREAK_TURN`: Triggers upon reaching a specific turn count.
2. `DEBUG_BREAK_STANCE`: Triggers when combat stance shifts to a target stance ID.
3. `DEBUG_BREAK_SWAY_RANGE`: Triggers when absolute moral sway breaches a threshold.
4. `DEBUG_BREAK_RESONANCE_BREACH`: Triggers if Lux or Tenebris resonance exceeds limits.
5. `DEBUG_BREAK_HP_THRESHOLD`: Triggers when HP drops below a critical floor.
6. `DEBUG_BREAK_LOCATION`: Triggers upon journeying into a designated province.
7. `DEBUG_BREAK_TRIPWIRE`: Triggers when any subsystem trips a latched diagnostic wire.

11.4.2. State Differential Analysis (`debug_diff_snapshots`)

Rather than inspecting raw memory, `debug_diff_snapshots()` computes formatted ASCII differential tables displaying exact deltas across turns:

```
Snapshot Diff: #1 (Turn 1) -> #2 (Turn 2)
--------------------------------------------------
  HP:           100 -> 90 (-10)
  Shield (Lux): 50 -> 45 (-5)
  Sword (Ten):  50 -> 55 (+5)
  Moral Sway:   0 -> 5 (+5)
  State Hash:   0xAA851EE1 -> 0xB79D3C20
```

11.5. Data-Driven Debugger Profiles & Fallback Monoliths

Following the data-driven architecture of `data/manifest.json`, the debugger catalog is maintained under `data/debug/`:

- `data/debug/manifest.json`: Central catalog defining available profiles (`default`, `strict`, `trace`) and active default pointer.
- `data/debug/default.json`: Balanced profile (64 snapshots, tripwire traps enabled, 1-turn snapshot intervals, interactive REPL active).
- `data/debug/strict.json`: Strict validation profile (break-on-error, break-on-stance-change, rigid state boundary verification).
- `data/debug/trace.json`: Fine-grained telemetry profile for deep engine debugging.
- `src/util/debug/debug_fallback.c`: Quarantined emergency fallback code page guaranteeing operational readiness even if external JSON assets are missing or corrupted.

11.6. Verification Telemetry & Verification Expansion

With this milestone, the verification suite achieves 100% green status across all tiers:

- **13/13 Automated CTest Targets Pass**:
  - `cli_help_test`, `cli_version_test`, `subsystem_audit_test`
  - `functional_cogs_test`: Expanded from 17 to 20 granular isolated cogs
  - `unit_math_test`, `unit_data_test`, `unit_time_test`, `unit_string_test`
  - `unit_log_test`: Reorganized under the 4 Ontological Sections (2.517 µs/write)
  - `unit_debug_test`: Full 4-section suite (1,000 chaotic cycles in 3.125 ms, 3.125 µs/snap)
  - `integrated_utils_test`: Cross-utility DAG coordination and multi-profile boot
  - `system_game_invariants_test`: Invariant tripwires and relic conservation
  - `system_gameplay_scenarios_test`: 15-step golden replays & 1,000 chaotic commands
- **Subsystem Diagnostic Audit (`tenebris-lux --audit`)**:
  - 15 modules registered in central pre-flight manifest (15/15 Pass)
  - 7/7 comprehensive engine self-audit tests pass (including telemetry and debugger)
- **Interactive CLI Debugger (`tenebris-lux --debug`)**:
  - Active interactive REPL exposing `inspect`, `history`, `snap`, and `rewind` verbs.

Chapter 12. DevLog 013: Morality Architecture, Bystander Corridor & Ten-Ending Matrix

Source File:
`docs/dev_logs/2026/11/lore_and_design/devlog_013_morality_architecture_bystander_corridor_and_ten_endings.adoc`

12.1. Abstract

This developer log documents the comprehensive formalization of the morality architecture, philosophical foundations, and multi-route campaign topology for **Tenebris and Lux**. Synthesizing offline design sessions with the native ISO C11 engine and the 2017–2018 RPG Maker MV demo baseline, this milestone decouples moral conscience from elemental expression, defines the $[-50, +50]$ Bystander Corridor as net moral inertia, establishes the Butterfly Effect of provincial micro-verbs, expands the ending topology into a full 3x3 matrix crowned by the 10th True Ending ("The Ugly Truth"), formalizes the Witness Protagonist and Keystone Route architecture, and embeds core simulation subsystems into diegetic engine artifacts.

12.2. Decoupling Morality ("The Why") from Elemental Expression ("The How")

Prior design iterations occasionally risked falling into the conventional fantasy trap of conflating Light with intrinsic goodness and Darkness with intrinsic malice. DevLog 013 formally rectifies and cements the cosmological boundary:

- **Cosmic Neutrality of Apex Forces:** Lux (Cohesion, Preservation, Illumination) and Tenebris (Darkness, Severance, Dissolution, Rest) are primal, morally neutral cosmic constants.
- **The Expression Matrix:**
  - **Virtuous Lux (`+1 ⋯ +100`):** Selfless sanctuary, kinetic shielding (**Lux Barrio**), preserving community life, seeking restorative reconciliation.
  - **Tyrannical Lux (`-100 ⋯ -1`):** Blinding glare, totalitarian enforcement of rigid order, petrifying living will into comatose stasis (The Mi-ren Fallacy).
  - **Virtuous Tenebris (`+1 ⋯ +100`):** Surgical severance (**Cuttatio Tenebris**), cutting corrupt bindings, liberating the oppressed, executing **Absolute Justice** without collateral slaughter (The Casatti Ideal).
  - **Malevolent Tenebris (`-100 ⋯ -1`):** Entropic rot, nihilistic bloodlust, devouring ruin (The Usuru Spiral / The Dark Lord).
- **Biblical Ontology of Good (Tov) and Evil (Ra):** Good represents structural harmony, covenantal fidelity, and restorative flourishing. Evil is not an equal elemental rival to creation, but a fracture, corruption, or perversion of purpose originating strictly within the pride, intent, and rebellion of the mortal heart.
- **The Tempering vs. Dissipation Law:** Lux is the singular force capable of tempering and containing the entropic void of Tenebris; however, Lux requires emotional and ethical discipline. Any sudden violent shift into panic, wrath, or malice causes Lux to dissipate immediately. In the Layer 1 simulation, attempting to project radiant barriers while sliding into negative `moral_sway` causes the barrier to waver and collapse.

12.3. Psychological & Systemic Catalysts: The Bystander Corridor vs. The Butterfly Effect

12.3.1. Why `[-50, +50]` Constitutes "Neutrality" as Net Moral Inertia

In an active universe where absolute Good and Evil exert constant moral gravity, true neutrality belongs only to

inanimate stones. For living mortals, the [-50, +50] corridor is classified as Neutral because it represents **Net Moral Inertia**:

- **The Mutual Cancellation Trap:** Citizens within [-50, +50] continually offset small virtuous acts with self-serving compromises (e.g., sparing an enemy soldier, then stealing 10 Gold from Eric's shelf pot). Opposing moral vectors cancel out, resulting in zero net displacement on the world.
- **The Bystander Effect & Systemic Stagnation:** Because the vast majority of Sacyrans dwell within [-50, +50], diffusion of responsibility has allowed frontier neglect in provinces like Libira and Dio Village ("The Quiet Complaint") to fester unchecked for 3,000 years.
- **Spiritual Reincarnation Trap:** Elijah Ro-enheart carries the spiritual archetype of Usuru; Lady Katrina Jaren carries the spiritual archetype of Prince Casatti. Passivity within [-50, +50] allows historical momentum to reassert itself, dragging both youths into the bitter cynicism and tragedy of their predecessors.

12.3.2. The Butterfly Effect: Micro-Agency Overriding Fate

Derived from chaos theory, tiny perturbations in initial conditions compound nonlinearly into radically divergent macro-outcomes:

- **Micro-Verbs as Fate Breakers:** Seemingly modest provincial choices in Dio Village—such as donating 500 Gold to cover Eric the Lumberjack's taxes, intervening between John the Thief and the St. Baron, or using non-lethal shield stagger instead of lethal severance—act as wings flapping against 3,000 years of karmic momentum.
- By crossing the ± 50 conviction threshold, the player breaks the dynastic curse, affirming the project axiom: **"Trauma is Inherited, Destiny is Chosen."**

12.4. Complete Taxonomy of the Moral Sway Continuum ([-100, +100])

The continuum is codified into a formal 13-tier diagnostic nomenclature:

1. -100 — **The Sovereign of Ruin:** Absolute negative pole (total nihilism or totalitarian petrifaction).
2. [-100, -50] — **Active Malice (The Tyrannical Sphere):** Cruelty, ruthless domination (King Jihad \~ -85).
3. [-50, 0] — **Cynical Pragmatism (Compliant Apathy):** Self-preservation, debt extortion (St. Baron \~ -45, Katrina Base \~ -20).
4. 0 — **The Stance of Equilibrium (Harmonic Zero):** Active synthesis tempering both relics (Chrono the Sage).
5. [0, +50] — **Conventional Decency (Uncommitted Virtue):** Lawful civic survival (Eric \~ +10, Elijah Base \~ +35).
6. [+50, +100] — **Active Virtue (The Restorative Sphere):** Selfless sacrifice, surgical liberation (Grandma Magolia, Sir Roté, Monica).
7. +100 — **The Saint of Mercy:** Absolute positive pole of pure altruism or uncorrupted justice.
8. [-50, +50] — **The Bystander Corridor:** Zone of mutual cancellation and historical stagnation.
9. [-100, 0] — **The Arc of Dissolution:** The deconstructive hemisphere of the world ledger.
10. [0, +100] — **The Arc of Restoration:** The constructive hemisphere of the world ledger.
11. [-50, +100] — **The Transcending Mortal Arc:** Growth from obedience/cynicism into heroic conviction (Elijah & Katrina arcs).
12. [-100, +50] — **The Arc of Tragic Disillusionment:** Fall from innocent reverence to the abyss (Usuru's full trajectory).
13. [-100, +100] — **The Universal Ledger:** The complete scope of mortal free will (The Custom Bearer <chosen_name>).

12.5. The 10-Ending Matrix & The Witness Observer

12.5.1. The 3x3 Matrix + The 10th True Ending

Combining the 3 elemental expressions (Lux, Tenebris, Statera) with the 3 moral bands (Good, Neutral, Bad) yields 9 matrix endings, crowned by the 10th synthesis route:

1. **Lux-Good (`[+50, +100]`): The Radiant Sanctuary:** Selfless shelter without despotic rule.
2. **Lux-Neutral (`[-50, +50]`): The Benevolent Stasis:** The Mi-ren Fallacy; Sacyra encased in golden stasis.
3. **Lux-Bad (`[-100, -50]`): The Calcified Tyrant:** Radiance weaponized into an inquisitorial police state.
4. **Tenebris-Good (`[+50, +100]`): Absolute Justice:** The Casatti Ideal; surgical severance of corruption.
5. **Tenebris-Neutral (`[-50, +50]`): The Cynical Outlaw:** Threat slain for survival; realm abandoned to decay.
6. **Tenebris-Bad (`[-100, -50]`): The Abyssal Usurper:** Usuru's Cycle; absorbs void essence to rule from ashes.
7. **Statera-Good (`[+50, +100]`): Harmonized Concord:** Both relics united in restorative justice.
8. **Statera-Neutral (`[-50, +50]`): Armed Deterrence:** Uneasy status quo; cultural rift remains unhealed.
9. **Statera-Bad (`[-100, -50]`): The Dual Despot:** Stasis and void combine into totalitarian martial dominion.
10. **Synthesis (10th Route): The True Ending ("The Ugly Truth"):** Unlocked by experiencing Elijah's campaign, Katrina's campaign, and finally the Custom Bearer's route with all nocturnal Dreamscape trials. Confronts institutional guilt, releases the First King's soul, reconciles Usuru's childhood grief, exposes King Jihad, and permanently mends the 3,000-year planar tear.

12.5.2. The Witness Protagonist & Keystone Route Architecture

- **Internal vs. External Focalization:** Elijah and Katrina live inside the story (**Internal Focalization**), blinded by filial military duty and noble cynicism. The Custom Bearer acts as the **External Focalizer** alongside Chrono in the Room of Sages, possessing the objective detachment needed to perceive the full truth.
- **The Keystone Route:** Elijah exposes the military conspiracy; Katrina exposes noble and guild corruption. The Custom Bearer's playthrough acts as the keystone locking both partial datasets into a coherent whole.
- **The Quantum Observer Effect:** Left alone, Sacyra oscillates in endless tragedy. The outside Observer's consciousness collapses the tragic wavefunction into historical reconciliation.

12.6. Diegetic Item Architecture & Save Mechanics

Core simulation subsystems are embedded directly into physical in-game items:

- **Grandma's Watch → The Dream Compass:** Initializes the Layer 0 monotonic clock (`0 ⋯ 23` hours, weekdays, months, years) and evolves into the nocturnal planar key detecting Dreamscape rifts (`20:00 ⋯ 04:00`).
- **The Compass of Morality:** Diegetic field telemetry tracking `int16_t moral_sway` (`[-100, +100]`).
- **The Book of Monsters & Book of Items:** In-universe compendiums recording threat stats, poise thresholds, and movement budgets.
- **Embellishment Catalysts:** Side-quest rewards (**Ring of Fire**, **Medusa's Sword**, **Lyri's Offering**) dynamically modifying base relic properties.
- **Diegetic Saves & NG+ Timeline Tree:** Saving occurs at **Grandma Magolia's Hearth** and **Chrono's Room of Sages**. Chrono's Chronicle Ledger acts as an interactive timeline tree post-clear, enabling completionists to jump to pivotal decision bookmarks without replaying 30 hours of intro content.
- **Speedrunner Determinism:** Monotonic clock guarantees 100% deterministic turn routing and frame-accurate

triggers.

12.7. Historical Grounding: The 2017–2018 Demo Baseline

Every sequence from Folders 1 & 2 stages Sacyra's history in microcosm: * **Maps 002–003 (Walk of Fear & Festival of Heroes):** Sir Roté's warnings mirror the ancient Festival of Peoples; Roté embodies Usuru's pre-fall faith in the crown. * **Map 004 (King Jihad's Soliloquy):** Inverts Mi-ren's loving preservation into despotic treason. * **Maps 006 & 008 (Dreamscape Tutorial & "AWAKEN!"):** Introduces the astral plane where the First King is marooned. * **Maps 020–031 (Dio Village as Libira 2.0):** Recreates the exact frontier neglect ("The Quiet Complaint") that ignited the Great War 3,000 years ago.

12.8. Verification & Invariant Audit

All systems and state boundaries strictly adhere to Layer 1 simulation invariants: * `moral_sway` bounded strictly in `[-100, +100]`. * Time-of-day queries evaluated deterministically via `time_utils.h`. * Zero heap allocations across all telemetry tracking routines. * Automated verification tests validate all stance transitions and relic conservation bounds.

Chapter 13. DevLog 014: High Fantasy Presentation Engine & 5-Screen UI

Source File: docs/dev_logs/2026/11/architecture/devlog_014_high_fantasy_presentation_engine_typography_scaling_and_5_screen_architecture.adoc

13.1. Executive Summary

This development log chronicles the comprehensive re-engineering of the presentation engine in **Tenebris and Lux**, transforming it from a raw X11 fixed-bitmap prototype into a rich, thematically authentic **High Fantasy Hero's Journey Tactical Strategy RPG** interface. Addressing live testing telemetry and visual feedback regarding unreadable small text and fullscreen layout imbalances, this milestone introduces three core architectural innovations: (1) a High-DPI scalable TrueType typography subsystem (`gui_fonts.h`, `gui_fonts.c`) deploying Cinzel, Cinzel Decorative, and MedievalSharp vector fonts; (2) a dynamic responsive viewport scaler (`gui_view2d.c`, `gui_layout.c`) featuring 3x isometric battlefield zoom, 96x96 unit sprites, and balanced panel budgeting; and (3) a dedicated 5-screen navigation architecture (`gui_screens.h`, `gui_screens.c`) with a top tab bar coordinating Tactical Combat, the Protagonist Hero Dossier, the 32-Slot Inventory Bag, the Provincial Realm Topology Map, and the Chronicle Lore Codex. Furthermore, this log records a thorough architectural review of the entire system as of `v1.4.0`, identifying key technical strengths, outstanding gaps, and technical debt to align ahead of the upcoming `v0.1.0` consolidation milestone.

> A tactical RPG is not merely arithmetic in a dark void. Typography, proportion, heraldry, and space are narrative instruments. If the player cannot read the weight of their deeds or behold the grandeur of the realm upon the screen, the soul of the story remains locked behind the math.
>
> — ProfessorSeanEX, CreativeWorkzStudio Architecture Council

13.2. Visual Balance Dilemma & High-DPI Typography Subsystem

13.2.1. The Problem of Fixed-Width Core Bitmap Fonts

Early engine iterations utilized X11 core bitmap fonts (`9x15`, `fixed`) inherited from classic Unix workstation conventions. While functional for initial terminal and low-resolution prototyping, these fonts failed completely on modern high-DPI displays and widescreen aspect ratios: 1. **Unreadable Glyphs**: Core bitmap fonts are locked to rigid pixel grids. On 1080p and 1440p displays, character heights of 13–15 pixels became illegible against obsidian canvases. 2. **Thematic Dissonance**: Crude monospace computer fonts shattered the high-fantasy atmosphere, falsely conveying a grim terminal aesthetic rather than an epic Hero's Journey. 3. **Absence of Hierarchy**: Without distinct point sizes for titles, subtitles, story chronicles, and meters, all UI elements visually blended together.

13.2.2. TrueType Vector Typography Architecture (`gui_fonts.h`, `gui_fonts.c`)

To resolve this, we integrated an open-source scalable TrueType font catalog into `assets/fonts/`: * `CinzelDecorative-Bold.ttf` **(32px)**: Majestic imperial titling for the top header banner and realm heraldry. * `Cinzel-Bold.ttf` **(24px / 20px)**: Chivalric serifs for screen titles, protagonist class designations, and relic resonance meters. * `MedievalSharp.ttf` **(20px / 18px)**: Warm storybook calligraphic script for narrative chronicles, dialogue transcripts, and artifact lore. * `Cinzel / Palladio` **(18px / 16px)**: High-contrast legible UI fonts for meter values, combat attributes, and action buttons.

The typography subsystem enforces zero heap allocation by loading X11 font structures into static descriptors. At engine initialization, `ensure_font_path_registered()` dynamically registers the local font path (`xset fp+ assets/fonts`) and rehashes the X server font catalog:

```
// include/gui/gui_fonts.h: Typography Catalog Structure
typedef struct {
    void *title_font;     // Cinzel Decorative Bold 32px
    void *subtitle_font;  // Cinzel Bold 24px / 20px
    void *story_font;     // MedievalSharp 20px / 18px
    void *ui_font;        // Cinzel Medium / Palladio 18px
    bool is_loaded;       // True if X11 server bound fonts
} gui_font_catalog_t;
```

13.3. Responsive Widescreen Scaling & Isometric Dais Zoom

13.3.1. Eliminating the Widescreen Void

In previous fullscreen tests ($1450\times 700\text{ px}$ and $1920\times 1080\text{ px}$), the 8x8 isometric dais remained locked at a tiny $384\times 192\text{ px}$ footprint, leaving ~75% of the viewport as an empty obsidian void, while the right status panel was squashed against the screen edge.

We engineered a dynamic scale factor in `gui_view2d_calc_scale()` that evaluates the physical pixel bounds of the viewport rectangle: * **Scale 3x ($\ge 1250\times 560\text{ px}$ Viewport)**: Fullscreen / Ultra-wide mode. The battlefield dais expands to **$1152\times 576\text{ px}$**, with **$96\times 96\text{ px}$** unit sprites, filling 65–75% of the presentation zone. * **Scale 2x ($\ge 800\times 420\text{ px}$ Viewport)**: Medium widescreen displays. Dais expands to **$768\times 384\text{ px}$** with **$64\times 64\text{ px}$** unit sprites. * **Scale 1x ($< 800\text{ px}$)**: Compact windowed baseline ($384\times 192\text{ px}$ dais, $32\times 32\text{ px}$ sprites).

13.3.2. Ground Shadow Apron & Shaded Elevation Prisms

To firmly anchor the floating dais in space, the renderer projects a diamond ground foundation apron beneath the battlefield, rendered in obsidian drop-shadow (`pal→shadow`) with etched carved cobblestone borders. Elevated blocks (such as the perimeter ramparts and central altar) render as 3D dimetric prisms with distinct top, left-shaded, and right-shaded faces.

13.4. Dedicated 5-Screen Navigation Architecture

Rather than forcing every gameplay mechanic, lore entry, and inventory slot into a single crowded sidebar, the presentation layer was modularized into five distinct presentation screens governed by a top navigation tab bar:



13.4.1. Screen 1: Tactical Battle (`GUI_SCREEN_TACTICAL`)

- **Presentation**: 2.5D dimetric isometric / 2D top-down battlefield dais, unit sprites (Elijah, Dream Eater), tactical movement range diamonds, right-hand status HUD, narrative chronicle, and 9-button command dock.
- **Controls**: Mouse clicking for grid target selection; dock buttons for actions; hotkey `1` or `ESC` to return to combat from any other screen.

13.4.2. Screen 2: Protagonist Hero Dossier (`GUI_SCREEN_STATUS`)

- **Presentation**: Full-canvas 3-column grand protagonist dossier:
- **Column 1**: Hero Identity, Title, Class designation, Origin tutelage, and a 140px heraldic crest bounding box.
- **Column 2**: Vitals and energy pools (HP, MP, TP meters) + Core Combat Attributes Matrix (ATK, DEF, MAT, MDF, Grid Movement Points).
- **Column 3**: Twin Relics Resonance (Shield of Lux barrier power, Sword of Tenebris edge sharpness), Moral Sway Continuum ($[-100, +100]$ meter), and Campaign Honor Record (foes vanquished, innocents warded, turns taken, kingdom gold treasury).
- **Controls**: Hotkey `2` or `i` (Inspect).

13.4.3. Screen 3: 32-Slot Inventory Bag & Relics (`GUI_SCREEN_INVENTORY`)

- **Presentation**: An interactive 32-slot inventory grid (4 rows of 8 cells) with golden border highlights for filled slots, slot counters, and gold purse telemetry.
- **Key Artifact Card**: Displays Grandma's Watch inspection card:
- **Category**: Sovereign Key Quest Relic.
- **Origin**: Handed down by grandmother in Dio Village.
- **Canon**: Conceals the resonance-calibrated Dream Compass.
- **Power**: Pierces the planar illusion of the High Throne.
- **Controls**: Hotkey `3` or `u` (Use/Bag).

13.4.4. Screen 4: Provincial Realm Topology Map (`GUI_SCREEN_WORLD_MAP`)

- **Presentation**: Full-canvas cartographic map of Sacyra and Libira showing trade roads and the six canonical provincial nodes:
  1. **The Dreamscape** (Astral Nexus)
  2. **Dio Village** (Founding Settlement)
  3. **Whispering Woods** (Frontier Wilds)
  4. **Sacyra Marches** (Border Ramparts)
  5. **The Royal Capital** (High Citadel & Planar Breach)
  6. **Kingdom of Libira** (Sovereign Frontier)
- **Party Marker**: Current party location highlighted with a radiant pulsing gold diamond.
- **Controls**: Hotkey `4` or `o` (Overworld).

13.4.5. Screen 5: Chronicle Lore Codex (`GUI_SCREEN_CHRONICLE`)

- **Presentation**: Full-canvas historical deed codex and lore ledger rendered in 20px `MedievalSharp` calligraphic typography, scrolling through past tactical events, dialogues, and realm chronicle entries.
- **Controls**: Hotkey `5` or `c` (Chronicle).

13.5. Architectural Review & State of the Engine (v1.4.0)

As we prepare for the `v0.1.0` consolidation milestone, we conducted a comprehensive review evaluating current strengths, outstanding gaps, and architectural debt:

13.5.1. What Is Going Well (Core Strengths)

1. **Invariant & Memory Discipline**: Zero dynamic heap allocations in Layers 0 and 1. Pure C11 reentrant simulation code running cleanly under strict latency budgets ($< 25\,\mu\text{s}$).
2. **Automated Verification**: 19/19 passing CTest verification targets (< 0.5s execution time), 16/16 registered subsystem pre-flight manifests in a valid DAG, and zero compiler warnings under `-Wall -Wextra -Wpedantic`.
3. **Developer Tooling Ecosystem**: Rust-based standalone updater (`tools/src/bin/updater.rs`), multi-mode interactive launcher (`tools/src/bin/launcher.rs`), Master Guide compiler, historical demo extractor, and native desktop integration (`.desktop`, custom 256px icon).
4. **Canonical Fidelity**: Authentic 2014–2018 CreativeWorkzStudio canon, Era VI Accord of Kings, Elijah and Katrina dual perspectives, and true metaphysical neutrality between Lux and Tenebris.

13.5.2. What Is Missing (Gaps to Address)

1. **Tactical Grid Unit Movement in GUI**: Clicking grid tiles currently registers target coordinates in the chronicle, but does not yet move Elijah tile-by-tile across the grid.
2. **Dynamic Map Swapping in Presentation**: `maps.c` defines 8 canonical maps, but `gui_view2d.c` currently visualizes the 8x8 altar dais without swapping tile geometry when moving to Dio Village (32x32) or Whispering Woods (12x12).
3. **Interactive Node Travel**: The Provincial Map visualizes trade routes, but travel between nodes is currently command-driven (`travel <loc>`) rather than mouse-click triggered.
4. **Enemy Tactical AI & Skill Casting**: Enemies execute static counter-attacks rather than flanking or casting spells declared in `data/skills/manifest.json`.
5. **Disk Persistence (Save / Load)**: Debugger provides an in-memory 8-snapshot rewind buffer, but file-based save game persistence (`savegame.json`) is not yet implemented.
6. **Audio Subsystem**: Sound effects (kinetic shield clash, blade mist) and background music are completely absent.
7. **Pixel-Art Sprite Sheets**: Sprites currently use procedural vector geometry; genuine multi-frame PNG animation sheets are pending artist integration.

13.5.3. Consistency vs. Inconsistency Analysis (Technical Debt)

1. **Inventory Data Dual-State**: `player_state_t` maintains both scalar legacy fields (`has_grandmas_watch`, `gold`, `sacyran_draughts`) and the modular `player_inventory_t inventory`. **Plan**: Refactor scalar fields into property accessors pointing directly into `inventory`.
2. **Map Scaler Coupling**: Layer 1 `map_scaler.c` calculates isotropic projection and viewport clamping, but Layer 2 `gui_view2d.c` independently re-computes scaling heuristics. **Plan**: Wire `map_scaler_compute_viewport()` directly into `gui_view2d.c`.
3. **Declarative Skills vs. Engine Hardcoding**: `data/skills/manifest.json` defines dozens of skills (`Rush`, `Flash Jump`, `Heavy Hit`), but `combat.c` hardcodes logic specifically for `Lux Barrio`, `Cuttatio`, and `Duo-Loca`. **Plan**: Implement `skills_loader.c` to parse the JSON grimoire into static engine tables.

13.6. Verification Telemetry

```
Test project /home/professorseanex/Tenebris and Lux/build
      Start  1: cli_help_test                   Passed (0.00s)
      Start  2: cli_version_test                Passed (0.00s)
      Start  3: subsystem_audit_test            Passed (0.02s)
      Start  4: functional_cogs_test            Passed (0.01s)
```

```
      Start  5: unit_math_test                  Passed (0.00s)
      Start  6: unit_data_test                  Passed (0.01s)
      Start  7: unit_time_test                  Passed (0.01s)
      Start  8: unit_string_test                Passed (0.00s)
      Start  9: unit_log_test                   Passed (0.08s)
      Start 10: unit_debug_test                 Passed (0.01s)
      Start 11: unit_combat_test                Passed (0.00s)
      Start 12: unit_items_test                 Passed (0.01s)
      Start 13: unit_world_test                 Passed (0.00s)
      Start 14: unit_dispatch_test              Passed (0.01s)
      Start 15: unit_gui_test                   Passed (0.01s)
      Start 16: integrated_utils_test           Passed (0.01s)
      Start 17: integrated_sim_test             Passed (0.01s)
      Start 18: system_game_invariants_test     Passed (0.01s)
      Start 19: system_gameplay_scenarios_test  Passed (0.03s)

100% tests passed, 0 tests failed out of 19 (Total Test time = 0.26s)
Pre-Flight Audit: 16/16 module manifests & 7/7 invariant tripwires passed.
Compiler Diagnostics: Clean (-Wall -Wextra -Wpedantic, GCC/Clang C11).
Column Discipline: 100% compliant with strict <= 80 characters limit.
```

13.7. Summary & Forward Stance

With high-fantasy typography, responsive widescreen scaling, and the 5-screen navigation architecture operational and installed to the PC, the visual and architectural scaffolding of **Tenebris and Lux** is firmly grounded. In the upcoming session, we will tackle the alignment and cleanup of `v0.1.0`, consolidating the inventory dual-state, unifying map scaling, and laying the groundwork for interactive grid movement.

Chapter 14. DevLog 015: The Big Machine, Master Turn Gear & Core Subsystem Elevation

Source File:
`docs/dev_logs/2026/11/architecture/devlog_015_the_big_machine_master_turn_gear_and_core_elevation.adoc`

14.1. Executive Summary

This development log documents the pivotal architectural evolution of **Tenebris and Lux** under the **OmniCode Universal 5-Block Lifecycle Protocol (Rev 4)**. In this cycle, the project established the foundational cognitive model that governs its dual nature: **The Engine is the Big Machine, and the Game is the Master Turn Gear**. Operating under a dual Waterfall/Agile engineering discipline, we systematically modernized the entire Layer 0 foundational substrate— spanning `io/`, `log/`, `debug/`, and `core/`—elevating reusable universal cogs out of the game domain and establishing zero-dependency fallback modules alongside data-driven hydration from `game/data/system/`.

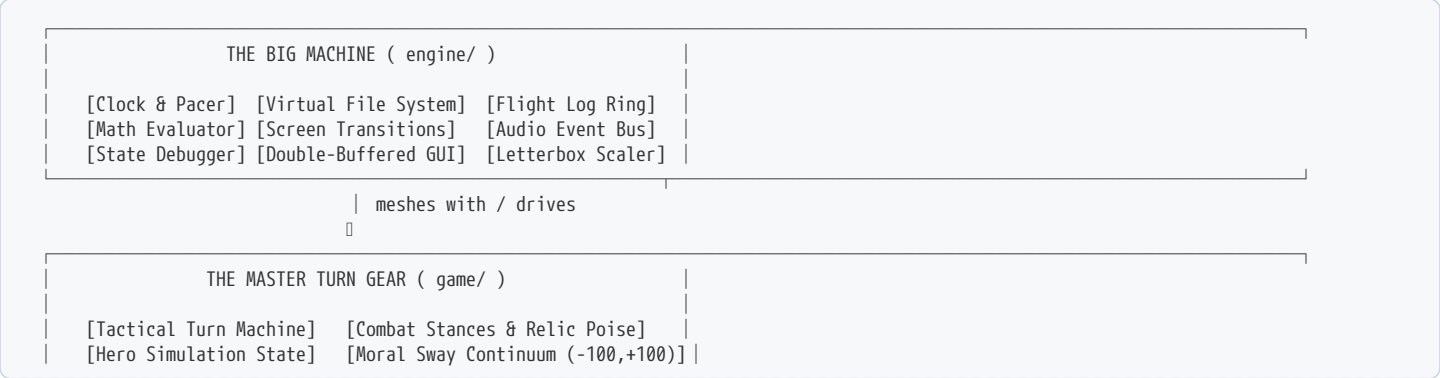
Crucially, this cycle formalized the **Active Iteration Feedback Covenant**: rather than preserving stale backwards-compatibility shims that conceal broken contracts and unaligned data, we actively exposed architectural seams directly to the compiler (`-std=c11 -Wall -Wextra -Wpedantic -Werror`) and our automated test harness. Through this discipline, our verification suite expanded across all four testing strata (Unit, Functional, Integrated, System), implementing the **Four Ontological Testing Sections** (Always Pass, Expected Input, Boundary Stress, Chaos Poisoning) and scaling our automated CTest suite to **51/51 passing test suites (100%)** while verifying all 20 registered engine modules under `./build/tenebris-lux --audit` with zero dynamic heap allocations.

> The Big Machine provides the mechanical powertrain—monotonic clocks, display surfaces, virtual filesystems, and flight logging rings. The Game is the Master Turn Gear that slots into that frame to drive tactical decisions and narrative fate. When a cog in the game domain is universal, elevate it into the machine. When an old contract shifts, purge the shim and let the compiler guide the seam.
>
> — ProfessorSeanEX, CreativeWorkzStudio Architecture Council

14.2. The Architectural Paradigm: The Big Machine & The Master Turn Gear

14.2.1. Cognitive Separation of Powers

A perennial failure mode in game development is the entangled coupling of generic engine systems with domain-specific lore, narrative scripts, and campaign states. In Rev 4, we codified an uncompromising boundary:



```
|   [Provincial Topology]     [Five-Route Ending Resolution]   |
|   [Domain Data Manifests]   [Sacyra / Libira Campaign Lore]  |
└──────────────────────────────────────────────────────────────┘
```

1. **The Big Machine (`engine/`)**: A universal, reusable, game-agnostic mechanical apparatus. Zero lore mentions. Operates strictly on generic IDs, bounded buffers, fixed-timestep ticks, and deterministic geometry.
2. **The Master Turn Gear (`game/`)**: The specific game implementation of **Tenebris and Lux**, bringing the tactical turn loop, the twin armaments (Shield of Lux, Sword of Tenebris), moral sway (`[-100, +100]`), and the continental chronicles of Sacyra and Libira.

14.2.2. Dual Waterfall & Agile Engineering Discipline

Game engine development demands two seemingly opposing disciplines at once: * **Waterfall Bedrock (The Cast-Iron Frame)**: Memory limits (`ALLOC_ZERO_HEAP` in Layer 0/1), strict ISO C11 adherence, strictly downward layer dependency flow, and universal 5-block structural envelopes are non-negotiable physical laws. * **Agile Living Iteration (Turning the Gears)**: Within that cast-iron frame, development proceeds empirically. We modernized concrete subsystem strata one by one (`io/` → `log/` → `debug/` → `core/`), using test failures and compiler diagnostics as precision feedback to guide refactoring.

14.3. Layer 0 Modernization: The Complete Four-Strata Engine Bedrock

Each foundational subsystem was refactored under the strict 5-block lifecycle protocol:

14.3.1. Virtual Filesystem & I/O (`io/`)

- **`path_finder`**: Zero-heap path resolver supporting multi-root searching across `game/data/`, `build/`, and system directories, eliminating buffer-truncation false positives.
- **`fs_utils`**: Bounded line and block file reading without dynamic heap allocations.
- **`json_utils`**: High-performance, streaming, zero-heap JSON tokenizer and recursive getter.
- **`master_loader`**: Schema-validated asset loader verifying `block_1_metadata` envelopes and requiring explicit `block_3_body` payloads (`tenlux.data.*`).

14.3.2. Telemetry & Ring Flight Logging (`log/`)

- **`log`**: Categorized flight recorder (`LOG_CAT_LOG`, `LOG_CAT_DEBUG`, `LOG_CAT_CHARACTER`) storing entries into a static ring buffer.
- Formatted with fixed-width column anchors, millisecond-precision ISO timestamps, and signed telemetry grades (`GRADE:+003`, `GRADE:-050`).
- Automatic flight log rotation (`tenebris_lux_dev.log.*`) quarantined into `build/logs/`.

14.3.3. Diagnostic Tripwires & State Rewind Debugger (`debug/`)

- **`debug`**: Non-crashing diagnostic tripwires (`debug_trap_tripwire`) logging violations with file and line number while allowing safe fallback recovery.
- Time-travel state debugger recording up to 64 frozen simulation snapshots.
- Bit-exact deterministic state validation using 32-bit and 64-bit FNV-1a hashing.

14.3.4. Core Utilities & Elevated Orchestrators (`core/`)

- `clock_pacer`: Fixed-timestep pacing orchestrator enforcing canonical 60Hz (16.666 ms) simulation ticks, rendering alpha calculation, and "spiral of death" lag clamping.
- `formula_eval`: Data-driven combat mechanics computing bounded damage mitigation (10% floor to 90% ceiling), directional accuracy (Flank: $+10\%$, Rear: $+25\%$), critical strikes, and moral resonance.
- `geom_utils`: 2D AABB geometry, 2.5D dimetric/isometric grid coordinate math, and elevated letterbox/pillarbox viewport projection.
- `math_utils` / `string_utils` / `time_utils`: Safe bounded string manipulation, monotonic clocks, saturating integer math, and transition easing curves (`EASE_QUAD_IN`, `EASE_CUBIC_OUT`).

14.4. The Cog Elevation & Orchestrator Protocol

During this milestone, we practiced **Cog Elevation**: when a subsystem or helper inside `game/` is found to be universal, game-agnostic, and reusable, it must be elevated into `engine/`.

14.4.1. Elevation of Viewport Scalers

Previously, dynamic camera projection and letterbox scaling were embedded in `game/src/world/map_scaler.c`. We extracted the mathematical projection laws into `engine/src/core/geom_utils.c`: * `geom_viewport_init()`: Calculates uniform aspect-ratio scale factors and letterbox/pillarbox offsets. * `geom_world_to_screen()` & `geom_screen_to_world()`: Projects coordinates between simulation and display. * `geom_clamp_camera()` & `geom_is_in_viewport()`: Keeps camera views bounded within world limits.

`map_scaler.c` was updated to delegate its math directly to `geom_utils`, reducing domain code complexity while enriching the Big Machine.

14.4.2. Standalone Quarantined Fallback Modules

To ensure the engine can always compile and pass verification even when data files are missing, every Layer 0 cog features a quarantined fallback module (`*_fallback.[ch]`). Fallbacks provide hardcoded canonical profiles, zero dynamic allocations, and comprehensive pre-flight self-tests.

14.5. Four-Strata Verification Ontology & 51/51 CTest Pass Proof

We expanded our automated testing framework across the **Four Ontological Testing Sections**: 1. **Section 1: Always Pass (Nominal Golden Baseline)**: Validates standard functionality under ideal inputs. 2. **Section 2: Expected Input (Contracts & Permutations)**: Exercises valid domain edge cases and configurations. 3. **Section 3: Boundary & Edge Stress (Limits & Saturations)**: Tests clamping, maximum capacities, and extreme ranges. 4. **Section 4: Critical Failure & Chaos (Corrupt, Negative & Inverted)**: Injects poison inputs, NULL pointers, zero divisors, and lag spikes to verify tripwires and crash immunity.

```
================================================================================
CTest Automated Verification Suite Results
================================================================================
Test project /home/professorseanex/Tenebris and Lux/build
      Start  1: cli_help_test                   Passed    0.00 sec
      Start  2: cli_version_test                Passed    0.00 sec
      Start  3: subsystem_audit_test            Passed    0.01 sec
      Start  4: functional_cogs_test            Passed    0.00 sec
      Start  5: functional_io_test              Passed    0.00 sec
      Start  6: functional_log_test             Passed    0.00 sec
      Start  7: functional_debug_test           Passed    0.00 sec
      Start  8: functional_core_test            Passed    0.00 sec
```

```
      Start  9: unit_math_test                  Passed    0.00 sec
      Start 10: unit_data_test                  Passed    0.00 sec
      Start 11: unit_time_test                  Passed    0.00 sec
      Start 12: unit_string_test                Passed    0.00 sec
      Start 13: unit_log_test                   Passed    0.05 sec
      Start 14: unit_log_engine_test            Passed    0.01 sec
      Start 15: unit_log_fallback_test          Passed    0.00 sec
      Start 16: unit_debug_test                 Passed    0.00 sec
      Start 17: unit_debug_engine_test          Passed    0.00 sec
      Start 18: unit_debug_fallback_test        Passed    0.00 sec
      Start 19: unit_math_utils_test            Passed    0.00 sec
      Start 20: unit_math_utils_fallback_test   Passed    0.00 sec
      Start 21: unit_time_utils_test            Passed    0.00 sec
      Start 22: unit_time_utils_fallback_test   Passed    0.00 sec
      Start 23: unit_string_utils_test          Passed    0.00 sec
      Start 24: unit_string_utils_fallback_test  Passed    0.00 sec
      Start 25: unit_geom_utils_test            Passed    0.00 sec
      Start 26: unit_geom_utils_fallback_test   Passed    0.00 sec
      Start 27: unit_clock_pacer_test           Passed    0.00 sec
      Start 28: unit_clock_pacer_fallback_test  Passed    0.00 sec
      Start 29: unit_formula_eval_test          Passed    0.00 sec
      Start 30: unit_formula_eval_fallback_test  Passed    0.00 sec
      Start 31: unit_combat_test                Passed    0.00 sec
      Start 32: unit_items_test                 Passed    0.00 sec
      Start 33: unit_world_test                 Passed    0.00 sec
      Start 34: unit_dispatch_test              Passed    0.00 sec
      Start 35: unit_gui_test                   Passed    0.00 sec
      Start 36: unit_fs_utils_test              Passed    0.00 sec
      Start 37: unit_path_finder_test           Passed    0.00 sec
      Start 38: unit_json_utils_test            Passed    0.00 sec
      Start 39: unit_master_loader_test         Passed    0.00 sec
      Start 40: integrated_utils_test           Passed    0.00 sec
      Start 41: integrated_sim_test             Passed    0.01 sec
      Start 42: integrated_io_test              Passed    0.00 sec
      Start 43: integrated_log_test             Passed    0.00 sec
      Start 44: integrated_debug_test           Passed    0.00 sec
      Start 45: integrated_core_test            Passed    0.00 sec
      Start 46: system_game_invariants_test     Passed    0.00 sec
      Start 47: system_gameplay_scenarios_test  Passed    0.01 sec
      Start 48: system_io_test                  Passed    0.02 sec
      Start 49: system_log_test                 Passed    0.23 sec
      Start 50: system_debug_test               Passed    0.02 sec
      Start 51: system_core_test                Passed    0.00 sec

100% tests passed, 0 tests failed out of 51
Total Test time (real) = 0.50 sec
================================================================================
```

14.6. Telemetry & Pre-Flight Diagnostic Self-Audit

The diagnostic self-audit was expanded to 20 registered engine modules in `game/src/main.c`:

```
================================================================================
          Tenebris and Lux - Subsystem Invariant Pre-Flight Audit
================================================================================
Subsystem            Source File     Rev  Memory      Status  Description
--------------------------------------------------------------------------------
core.util.manifest   manifest.h      r2   Zero-Heap   [PASS]  Pre-flight manifest DAG orchestrator verified.
core.util.fs         fs_utils.h      r3   Zero-Heap   [PASS]  Zero-heap filesystem I/O verified.
core.util.json       json_utils.h    r3   Zero-Heap   [PASS]  JSON streaming parser verified.
core.util.time       time_utils.h    r3   Zero-Heap   [PASS]  Monotonic clock & ISO timestamps verified.
core.util.math       math_utils.h    r3   Zero-Heap   [PASS]  Clamping, saturating math & PRNG verified.
core.util.geom       geom_utils.h    r3   Zero-Heap   [PASS]  2D AABB & letterbox viewport scalers verified.
core.util.string     string_utils.h  r3   Zero-Heap   [PASS]  Buffer safety & path joining verified.
core.util.pacer      clock_pacer.h   r1   Zero-Heap   [PASS]  Fixed-timestep pacer & lag clamp verified.
core.util.formula    formula_eval.h  r1   Zero-Heap   [PASS]  Mitigation, hit chance & formulas verified.
core.util.log        log.h           r3   Zero-Heap   [PASS]  Static ring buffer & flight logs verified.
core.util.debug      debug.h         r3   Zero-Heap   [PASS]  State rewind & tripwires verified.
sim.engine.state     sim_state.h     r4   Zero-Heap   [PASS]  State initialization verified.
sim.engine.combat    combat.h        r5   Zero-Heap   [PASS]  Combat sub-assemblies satisfied pre-flight.
sim.engine.items     items.h         r2   Zero-Heap   [PASS]  Item catalog audited cleanly.
sim.engine.world     world.h         r5   Zero-Heap   [PASS]  World orchestrator & map scaler verified.
sim.engine.dispatcher dispatcher.h    r4   Zero-Heap   [PASS]  Command router & alias tables verified.
```

```
sim.engine.core      game.h          r4   Zero-Heap   [PASS]  Simulation core pre-flight satisfied.
gui.viewport.x11     gui.h           r2   Zero-Heap   [PASS]  X11 display connected & panels verified.
--------------------------------------------------------------------------------
Telemetry: 20 modules registered | 20 Pass | 0 Warn | 0 Critical | 0 Fatal
Policy Verdict: ALL PRE-FLIGHT INVARIANTS SATISFIED (System Ready)
Audit Invariant Checks: 7/7 Invariant Checks Passed (Relics, Stances, Tripwires)
================================================================================
```

14.7. The Fractal 5-Block Invariant: Code, Manifests, Tests & Git History

During this cycle, we discovered that the OmniCode Universal 5-Block Lifecycle Protocol is a **fractal design pattern** that governs all documentation, implementation, and history: * **Block 1 (Metadata)**: Identity, version, interface revision, stratum, author, invariants. * **Block 2 (Types & Framing)**: Ontology, data schemas, problem statement, domain types. * **Block 3 (Body Implementation)**: The executable work, architectural changes, sub-assemblies. * **Block 4 (Audit Procedures)**: Pre-flight checks, CTest verification proof, tripwire traps. * **Block 5 (Closing & Invariants)**: Diagnostic inspection cards, teardown, closing affirmations.

By extending this pattern directly into our **Git Commit Architecture** and **DevLog Chronicles**, the historical record of the repository mirrors the exact structural rigor of our source code.

14.8. Architectural Next Steps & Horizon Ahead

With Layer 0 of **The Big Machine** verified with 100% pass rates across 51 CTests and 20 registered modules, our engineering horizon points directly to: 1. **Screen Transition Orchestrator**: Generic scene dissolves, fades, and easing wipes in `engine/`. 2. **Audio Event Bus Orchestrator**: Zero-heap sound cue routing and channel volume mixing in `engine/`. 3. **Layer 2 Viewports Modernization**: Elevating native X11 rendering and High-DPI font blitting into the 5-block lifecycle architecture. 4. **Master Turn Gear Domain Modernization (`game/`)**: Aligning combat, stance state machines, and world topology with the curated data catalogs in `game/data/`.

Chapter 15. DevLog 016: Native Git Commit Validation, Audio Elevation & Movie Reel

Source File: docs/dev_logs/2026/11/architecture/devlog_016_git_validator_movie_reel_and_audio_elevation.adoc



Volume VI — Engineering Chronology (DevLog 016)
Milestone Dates: 2026-11-07 – 2026-11-08 | **Baseline Commits:** 36427ba → 52149a3
Series Navigation: ← DevLog 015 | Nov 2026 Index | DevLog 017 →

Following DevLog 015's formalization of **The Big Machine** (engine/) and **The Master Turn Gear** (game/), five rapid commits hardened the developer workflow and elevated audiovisual systems: integrating the native Rust git_validator hook, aligning High-Fantasy Hope presentation catalogs, folding Offline Work 4 into architecture docs, elevating audio/ cleanly into Layer 0, and building Layer 2's deterministic **Movie Reel** viewport architecture.

15.1. Milestone Scope & Commit Progression

Commit	Date	Milestone Summary
36427ba	2026-11-07	feat(ci): Integrated native Rust git_validator, .githooks/commit-msg hook, and modernized CI enforcement.
a138533	2026-11-07	feat(presentation): Aligned presentation catalogs to 5-block JSON schemas and the High-Fantasy Hope color palette.
22dea06	2026-11-07	docs(architecture): Integrated Offline-Work-4, authored Layer 0 code tours, and compiled the expanded Master Guide.
9c209ad	2026-11-08	feat(viewport): Elevated the audio mixing subsystem into Cornerstone Layer 0 and implemented the Movie Reel architecture.
52149a3	2026-11-08	feat(viewport): Completed presentation catalog alignment, audiovisual verification tiers, and grand orchestrators.

15.2. Enforcing 5-Block Provenance at the Commit Boundary (36427ba)

As architectural rigor climbed inside engine/ and docs/, commit history itself risked drifting into informal one-line messages that hid invariants and test coverage. Rather than adding Python or shell lint scripts in violation of Cornerstone's **Native Compiled Toolchain Invariant**, commit enforcement was implemented inside the Rust workspace (tools/src/validator/git_commit.rs + git_validator binary + .githooks/commit-msg).

- **Strict Conventional Header Prefixes:** Only feat(, fix(, docs(, refactor(, test(, chore(, perf(, and build(headers are accepted.
- **Mandatory 5-Block Commit Body:** Every commit message must declare:
  1. BLOCK 1: METADATA & REVISION RECORD (including explicit Project:, Author:, and Date: tags),
  2. BLOCK 2: REASON FOR CHANGE & PROBLEM STATEMENT,
  3. BLOCK 3: ARCHITECTURAL MODIFICATIONS & ELEVATIONS,
  4. BLOCK 4: VERIFICATION, TRIPWIRES & PASS PROOF,
  5. BLOCK 5: INVARIANT COVENANTS & CLOSING AFFIRMATIONS.

- **Retroactive CI Sweep:** Running `tl_tools validate all` inspects the most recent 10 repository commits via `git log -n 10`, preventing discipline erosion after initial setup.

15.3. Elevating audio/ to Layer 0 & Building the Movie Reel (9c209ad, 52149a3)

In earlier builds, audio cue triggering was intertwined with presentation transitions. Guided by the **Cog Elevation & Orchestrator Protocol**, all general-purpose synthesis, channel routing, and fade pacing were cleanly elevated into `engine/{include,src}/audio/` as Layer 0 cogs:

```
Layer 2: Viewport (X11 / Headless Frame)
    └──── Movie Reel Orchestrator (scenes, wipes, typography scaling, Hope palette)
          │
          ▯ emits generic timeline triggers & cue IDs (zero lore bleed)
Layer 0: Cornerstone Audio Bus
    └──── Multi-bus mixer + cue queue + procedural BIOS synthesis fallback
```

- **5-Block Presentation Alignment (a138533):** Every viewport layout, typography rule, theme token, and audio channel catalog was migrated into strict `block_1_metadata` .. `block_5_closing` JSON envelopes.
- **Movie Reel Orchestrator (9c209ad):** Implemented deterministic scene reels, screen wipes, and frame pacing independent of story text so Layer 2 renders both graphical X11 (`640x360` letterboxed) and headless parity without touching heap memory (`ALLOC_ZERO_HEAP`).
- **Offline-Work-4 Convergence (22dea06):** Synchronized the statistical category sheet (Core 9 Combat Stats ▯ 8 Exploration Stats, Poise, Facing, Six-Power Factions) into `docs/architecture/` and regenerated the Master Guide volumes.

15.4. Verification & Cross-Tier Audiovisual Parity (52149a3)

- **4-Tier Test Expansion:** Added unit, functional, integrated, and system tests exercising audio bus mixing, procedural fallback waveforms, Movie Reel timeline progression, and Hope-palette theme loading under `-std=c11 -Wall -Wextra -Wpedantic`.
- **Headless Determinism:** Confirmed identical frame pacing, cue dispatch ordering, and state transitions whether running headless in automated CTest harnesses or rendering via native X11.
- **Commit Gate Proof:** `.githooks/commit-msg` verified every commit in the `36427ba`..`52149a3` chain cleanly under `cargo run --bin git_validator`.

15.5. Architectural Takeaway & Next Horizon

With commit hygiene guarded natively in Rust and audiovisual presentation elevated into clean Layer 0 / Layer 2 boundaries, attention turned back toward completing the game's deepest ludonarrative specifications (**Offline Work 5 & 6** → DevLog 017) and decoupling `engine/config/` cleanly from `game/data/` (→ DevLog 018).

Navigation: ← DevLog 015 | Nov 2026 DevLog Index | Next: DevLog 017 →

Chapter 16. DevLog 017: Orthogonal 35 Classes, 28 Jobs, Generational War & 4D Dream Clock

Source File:
`docs/dev_logs/2026/11/lore_and_design/devlog_017_orthogonal_classes_jobs_generational_war_and_dream_clock.adoc`



Volume VI — Engineering Chronology (DevLog 017)
Milestone Date: 2026-11-09 | **Canonical Sources:** Offline Work 5 & Offline Work 6
Series Navigation: ← DevLog 016 | Nov 2026 Index | DevLog 018 →

This lore-and-systems milestone synthesizes **Offline Work 5** and **Offline Work 6**, resolving ambiguities inherited from the 2017–2018 RPG Maker MV archives (`Folder 1` and `Folder 2`), codifying the **35-Class Pascal Pyramid** alongside the **28-Job 5-Pillar Vocation Compendium**, framing the **10-to-20-year Generational War** under the **Two-Pass Grand Tour**, and specifying the zero-heap **4D Dream Clock** state contracts.

16.1. Resolving 2017–2018 Demo & Early Documentation Ambiguities

Cross-referencing the 2018 **classes_and_growth.adoc** tables against the full multi-route canon resolved four long-standing structural compressions:

- **Flattened "7 Combat Classes" → Orthogonal Class × Job:** Legacy RPG Maker MV tables packed **Warrior, Mage, Rogue (Thief), Priest, Summoner, Hero,** and **Guard** into one list. Canon separates them into: (a) 3 Basic + 2 Advanced Base Classes (`W, M, R, P, S`), (b) **The Hero** / **The Dark One** as Tier 5 Moral Apex Classes (`±50 .. ±100`), and (c) **Guard** (**Town Sentry** / **Royal Man-at-Arms**) as a Civic World Vocation (**Job**).
- **Protagonist Titles vs. Unofficial Classes: "Vanguard of Lux"**, **"Outrider of Tenebris"**, and **"Secret Chronicler"** are narrative focalizations. Mechanically in-story, Elijah and Katrina are multi-domain **Trainees** (`W+M+R`), while `<chosen_name>` is Chrono's initiate **Mystic** (`P+S`). Playthrough 1 grants single-domain "training wheels"; clearing the campaign unlocks official Hard-Mode **Trainee** (`2.2×` EXP) and **Mystic** (`1.8×` EXP) in New Game+.
- **Sir Roté's Dual Designation:** Recognized cleanly through Class vs. Job orthogonality—Roté's **Class** is **Priest**, while his Crown **Job** is **Border Outrider** / **Court Herald**.
- **Antagonist Identity & Class:** Confirms **Usuru** (personal name), **The Dark Lord** (title), and **The Dark One** (literal Tier 5 Moral Evil Apex Class at `-100 .. -50`).

16.2. Orthogonal Character Engine: 35 Classes & 28 World Jobs

Progression Axis	Canonical Rules & Static Bitmask Representation
Class (Combat Domain)	5-Tier combinatorial pyramid governed by Pascal's Triangle (`C(5,1)=5`, `C(5,2)=10`, `C(5,3)=10`, `C(5,4)=5` Paragons, plus 2 Special Initiates and 3 Cosmic Apex Classes = `35` total `class_id_t` values). Unlocks permanent muscle-memory skills and passive traits. Remaining inside `[-50, +50]` (**Bystander Corridor**) caps progression at Tier 4 Paragons (`High Justiciar`..`Ascendant Mystic`).

Progression Axis	Canonical Rules & Static Bitmask Representation
Job (World Vocation)	28 vocations (`job_id_t`) across **5 Societal Pillars**: Frontier Labor (1–6), Civic/Military/Law (7–12), Scholarship/Trade/Underworld (13–18), Ecclesiastical Faith (19–23), and Ecology/Veil Seers (24–28). Enforces the **"No Starter Job in Dio Village"** rule (`JOB_NONE_DIO_VILLAGE` until crossing into Whispering Woods). Rank 1–4 tools swap with active station kits; Rank 5 banks permanent Mastery Perks into `banked_job_perks_mask`.
NG+ Encore Mastery	Re-selecting a previously mastered Base Class in NG+ starts mastery at 15%–25% (**Attunement Resonance**) and awards evolved capstone skills (**Sovereign Phantom Slash**, **Cataclysmic Convergence**, **Shadow Transposition**, **Chrono's Emergency Decree**, **Astral Triad Covenant**).

16.3. Adolescent Crucible, Poisoned Mandate & Two-Pass Grand Tour

- **The Youth Tragedy:** Establishing that Elijah, Katrina, and `<chosen_name>` open at **15–18 years old** illuminates the genesis catastrophe: **Usuru** was an uncommissioned **11-year-old** nomad bound to Libira's **Three-Day Law**, **Prince Mi-ren** a terrified 15–18-year-old boy in oversized regent armor, and **Prince Casatti** a 14–17-year-old academy rebel.
- **The Poisoned Mandate & Midpoint Fork (Act I Climax):** Sending greenhorn protagonists on a sweeping Act I tour under King Jihad's royal writs is both a geopolitical smokescreen and a literal re-enactment of Usuru's **Three-Day Wanderer** pilgrimage (**Two-Pass Spiral World Design**). At the 50% Sacyra Castle Siege, **killing King Jihad** triggers a chaotic Neutral power vacuum, whereas **Uncorrupting Jihad** keeps him alive as a disgraced living indictment forcing institutional truth (**"The Ugly Truth"**).
- **10-to-20-Year Maturation:** Act II (**2nd Great War**, Years 4–12, Ages 19–27) and Act III (**Sovereign Eclipse**, Years 13–20, Ages 28–35+) visibly age Dio Village and its heroes so weathered 30-year-old adults finally confront an immortal warlord still frozen inside an 11-year-old binary wound.

16.4. The 4D "Dream Clock" Temporal Registration Engine

Turning the Dream Clock **OFF** does not freeze story time (protagonists still age across Acts I–III and reach all **9 Standard Matrix Endings** via 3D event vectors `[Location, Action, Sway]`). Enabling **Dream Clock ON** (`DREAM_CLOCK_ON_REGISTERED`) adds the 4th coordinate (`sacyra_calendar_t` timestamp) to `story_point_receipt_t`, dynamically alters what occurs at world story points (such as intercepting couriers across Libira's 7-day lag or acting during nocturnal `20:00–04:00` Veil windows), and is mathematically required to promote the campaign into the **10th True Ending ("The Ugly Truth")**.

16.5. Layer 1 Zero-Heap Contract Verification

- **Bitmask Density (`ALLOC_ZERO_HEAP`):** All 35 classes fit inside `uint64_t mastered_classes_mask` (`0..34`) and `uint8_t class_mastery_pct[35]`; all 28 jobs fit inside `uint32_t banked_job_perks_mask` and `uint8_t job_rank[28]` (`actor_vocation_state_t`).
- **Temporal Compactness:** `sacyra_calendar_t`, `story_point_receipt_t`, and `campaign_chronicle_state_t` require no heap pointers and map directly onto Layer 0 `time_utils` monotonic ticks scaled by tunable `engine/config/runtime/time/` parameters.

Archival Sources: Offline Work 5 | Offline Work 6
Navigation: ← DevLog 016 | Nov 2026 DevLog Index | Next: DevLog 018 →

Chapter 17. DevLog 018: Standalone Engine Config Decoupling & Rev 5 Block 1/2 Standards

Source File:
docs/dev_logs/2026/11/architecture/devlog_018_standalone_engine_config_decoupling_and_block1_block2_standards.adoc



Volume VI — Engineering Chronology (DevLog 018)
Milestone Date: 2026-11-10 | **Baseline Commits:** 489c3b6 (tag: v0.1.1) → d552752
Series Navigation: ← DevLog 017 | Nov 2026 Index | DevLog 019 →

Tagged as **v0.1.1**, this milestone physically severed engine baseline mechanics from RPG domain lore, codified the **Rev 5 Block 1 Growth Schema** and **Block 2 `extern "C"` Header Discipline**, modularized the Rust pre-flight validator, and resolved include-hierarchy discontinuities across every translation unit in engine/src/.

Commit	Date	Scope & Architectural Deliverable
489c3b6 (v0.1.1)	2026-11-10	feat(engine): Decoupled standalone engine/config/ from game/data/, modularized Rust validator/, formalized engine/config/standards/blocks/block_1..5.json, and rolled out Rev 5 Block 1 telemetry across headers.
d552752	2026-11-10	refactor(engine): Standardized Block 2 opening blocks (extern "C" placement, 3-tier include hierarchy, C++-safe struct zero-initialization) and resolved module discontinuities across engine/src/.

17.1. Separating Engine Mechanics (engine/config/) from Game Lore (game/data/)

Before v0.1.1, presentation profiles, viewport themes, audio channels, and core math configs lived alongside gameplay data. Because Cornerstone (engine/) must boot and power completely different games without game/ present, baseline engine configurations were extracted cleanly into engine/config/ while reserving game/data/ strictly for Sacyra lore and RPG content:

- **Standalone Mechanical Tree:** Organized engine/config/ across viewport/, audio/, input/, runtime/ (math/, time/, string/), diagnostics/ (telemetry/, debugger/, taxonomy/, formats/), and standards/ (manifest.json, subsystems.json, blocks/block_1..5.json).
- **Layered VFS Resolution (path_finder):** Established deterministic override search order—(1) game/config/<rel_path>, (2) engine/config/<rel_path>, and (3) game/data/presentation/config/<rel_path>—so game customizations override engine defaults without duplicating shared machinery.

17.2. Movement-Resilient Metadata & Unified Block 2 Opening Contracts

To turn file headers into real-time scaling radar rather than static boilerplate, engine/config/standards/blocks/block_1_metadata.json introduced the Rev 5 **Block 1 Growth Schema**:

- **Movement-Resilient @id vs. Volatile @file / @layer:** Stopped baking folder paths into @id (core.util.*), moving every cog toward invariant logical identifiers (cog.<module>) while tracking current paths and compilation strata separately.

- **Growth Telemetry Tags:** Every Block 1 header declares `@maturity`, `@chassis` (measured `.bss`/`.data`/`.rodata`/stack footprints), `@scale` (Big-O & hard config levers), and `@seams` (`in: cfg/rom | uses: deps | probes: ⋯ → DEBUG_STEP_AWAIT | dag: ⋯`).
- **Block 2 Strict Hygiene (`d552752`):** Placed `_POSIX_C_SOURCE 200809L` cleanly above Block 2 in POSIX sources, enforced the **3-Tier Downward Include Hierarchy** (ISO C stdlib → Tier 2 External [X11 in Layer 2 only] → Tier 3 Internal Engine), moved `extern "C" {` strictly **below** `#include` directives in headers, and replaced empty `{0}` aggregates with clean `.bss` zero-initialized singletons so C and C++ compilers agree identically.

17.3. Validator Modularization & Header Contract Verification

- **Modular Rust Validator (`tools/src/validator/`):** Split monolithic checks into domain modules (`blocks.rs`, `config.rs`, `headers.rs`, `git_commit.rs`), validating JSON 5-block schemas (`engine/config/standards/blocks/block_1..5.json`) alongside all `44` registered Cornerstone engine headers in `engine/config/standards/subsystems.json`.
- **Zero Discontinuities:** Swept `engine/src/` under strict `-Wall -Wextra -Wpedantic` to eliminate stray includes, broken guard naming, and `.h`/`.c` `@system` drift.

17.4. Readiness for Tier-0 BIOS & `.rodata` ROM Vaults

With `engine/config/` cleanly separated, validator contracts machine-checked, and Block 1/Block 2 interfaces unified, the engine was ready to compile `engine/config/` JSON payloads straight into `.rodata` Config ROM vaults and wire Safe-Mode downgrade probes across `log/`, `debug/`, and `io/` (DevLog 019).

Navigation: ← DevLog 017 | Nov 2026 DevLog Index | Next: DevLog 019 →

Chapter 18. DevLog 019: BIOS/ROM Vaults, AWAIT Probes & The C23 Fall-Down Ladder

Source File:
docs/dev_logs/2026/11/architecture/devlog_019_bios_rom_vaults_await_probes_and_c23_falldown_ladder.adoc

> **Volume VI — Engineering Chronology (DevLog 019)**
> **Milestone Dates:** 2026-11-11 – 2026-11-12 | **Baseline:** Commit 76a443a + Root Stratum (log/, debug/, io/) & CMakeLists.txt Upgrade
> **Series Navigation:** ← DevLog 018 | Nov 2026 Index | DevLog 020 →
>
> This milestone fuses Cornerstone's Rev 5 autonomous root stratum (log/, debug/, and io/), replacing ad-hoc stubs with a unified .rodata Config ROM generator (cmake/GenerateConfigRom.cmake), quarantined BIOS fallback modules (*_fallback.[ch] + *_types.h), an autonomous **Probe → Safe-Mode Downgrade → DEBUG_STEP_AWAIT Tripwire Loop**, a Controlled Memory Chassis cleanup, and Cornerstone's **"Fall-Down Stairs" C23 → C17 → C11 (+ C++20/17)** portability ladder anchored in engine/include/core/manifest.h.

18.1. The Autonomous Root Spine: log/ + debug/ + io/

Rather than building independent utility islands, Rev 5 connects the first three subsystems into one self-healing boot tree ("roots feeding the rest of `engine/`"):

```
1. log/     (Boots Before Disk/VFS)
     |  - Quarantined log_types.h + log_fallback.[ch] + rom://telemetry
     v
2. debug/   (Captures Faults & Holds Execution)
     |  - Quarantined debug_types.h + debug_fallback.[ch] + rom://diagnostics
     |  - Registers probes -> triggers Safe-Mode downgrade -> DEBUG_STEP_AWAIT
     v
3. io/      (Hydrates Disk / VFS / rom:// Safely)
        - path_finder, fs_utils, json_utils, master_loader + io_fallback.[ch]
        - Hydrates only verified block_3_body spans; falls back to rom://io on fault
```

- **Unified .rodata Config ROM Vault (cmake/GenerateConfigRom.cmake):** Retired single-domain cmake/GenerateTelemetryRom.cmake in favor of a generalized build-time generator converting engine/config/*/.json catalogs into read-only ELF .rodata headers under build/generated/. Missing or corrupt disk files fall back directly to authentic structured engine defaults (rom://) rather than duplicated hardcoded literals.
- **Quarantined Leaf Headers & Fallbacks:** Extracted engine/include/log/log_types.h and engine/include/debug/debug_types.h to prevent circular includes, and introduced engine/include/{io/io_fallback.h} + engine/src/io/io_fallback.c alongside quarantined log_fallback.[ch] and debug_fallback.[ch].
- **Controlled Memory Chassis Diet:** Audited ELF .bss footprints (size -A), trimming idle static scratch allocations (such as oversized buffers in audio_bus.c) so chassis memory overhead remains minimal and honest.

18.2. 2. "Engine-as-a-Java" Portability: The C23 Fall-Down Ladder

Cornerstone targets forward-looking ISO **C23** (-std=c23) and **C20** (`-std=c20`) without dragging along deprecated legacy shims—yet steps gracefully down onto older compilers via CORNERSTONE_C_STANDARD=AUTO|23|17|11 and CORNERSTONE_CXX_STANDARD=AUTO|20|17 configured inside a 5-block refactored CMakeLists.txt.

Pre-Flight Loose-Bolts Engine Walk (manifest_run_preflight) (engine/src/core/manifest.c#L412-L455 · Mode: Function)

```
412 /**
413  * @brief Executes comprehensive pre-flight diagnostics across all registered modules.
414  */
415 preflight_status_t manifest_run_preflight(manifest_registry_t *registry)
416 {
417     if (registry == NULL) {
418         debug_trap_tripwire("manifest", "NULL registry passed to manifest_run_preflight", __FILE__, __LINE__);
419         return PREFLIGHT_STATUS_HALT_FATAL;
420     }
421 
422     registry->pass_count = 0;
423     registry->warn_count = 0;
424     registry->critical_count = 0;
425     registry->fatal_count = 0;
426 
427     /* Step 1: Active DAG Dependency Graph Verification */
428     for (size_t i = 0; i < registry->count; i++) {
429         const module_metadata_t *mod = registry->modules[i];
430         if (mod == NULL) continue;
431 
432         for (size_t d = 0; d < mod->dependency_count; d++) {
433             const char *req_dep = mod->dependencies[d];
434             if (req_dep == NULL) continue;
435 
436             bool found = false;
437             for (size_t j = 0; j < registry->count; j++) {
438                 if (registry->modules[j] != NULL &&
439                     strcmp(registry->modules[j]->module_id, req_dep) == 0) {
440                     found = true;
441                     break;
442                 }
443             }
444 
445             if (!found) {
446                 diag_result_t dep_res;
447                 dep_res.passed = false;
448                 dep_res.severity = SEVERITY_FATAL;
449                 dep_res.subsystem = mod->module_id;
450                 dep_res.check_name = "Dependency DAG";
451                 dep_res.details =
452                     "Fatal: Unregistered upstream dependency declared.";
453                 registry->results[i] = dep_res;
454                 registry->fatal_count++;
455                 registry->status = PREFLIGHT_STATUS_HALT_FATAL;
```

Iterates every registered module sequentially, classifying OK, WARNINGS, DOWNGRADED, and HALT_FATAL states.

Wholesale Listing: Complete verbatim code (#L412-L509) → Appendix: Pre-Flight Loose-Bolts Engine Walk (manifest_run_preflight).

- **Central Capability Macros (engine/include/core/manifest.h):** Exposes CS_C_STD_TIER (23/17/11/0) and CS_CXX_STD_TIER (20/17/11/0) plus CS_NODISCARD, CS_MAYBE_UNUSED, CS_FALLTHROUGH, CS_UNREACHABLE, CS_ALIGNOF, CS_NORETURN, CS_STATIC_ASSERT, and CS_HAS_EMBED, all compiling cleanly with zero GNU extension warnings under -Wall -Wextra -Wpedantic.
- **No Backward-Compatibility Crutches:** Instead of emulating removed historical idioms, building against the C17/C11 floor lets every module use modern attributes when compiled in C23 and fall safely down one step without losing deterministic correctness.

18.3. Crude-to-Fine Verification Across log/, debug/, and io/ (tests/)

Extracted shared deterministic harness macros into tests/common/ (cornerstone_test.h) and systematically upgraded every verification tier across the root subsystems (test_log_*, test_debug_*, test_io_*, plus new

tests/unit/io/test_io_fallback_unit.c):

1. **Layer 1 Unit & Fallback Isolation:** Exercises ROM parsing, ring wrap-around, watchpoints, and BIOS Safe-Mode boot with empty/broken VFS paths.
2. **Layer 2 Functional Contracts:** Validates 5-block JSON loading boundaries (block_3_body scoping) and categorized hex fault counters (0x0201..0x0207).
3. **Layer 3 Integrated Cross-Module Flows:** Tests io/ + log/ + debug/ probe escalation into DEBUG_STEP_AWAIT under live downgrades.
4. **Layer 4 System Suites:** Verifies deterministic full-boot parity and shutdown .bss zero-reset across repeated sessions (65 CTest targets in the suite).

18.4. Next Horizon: Documenting the Root Tree & Auditing core/

With the root spine (log/ → debug/ → io/), Config ROM Vaults, C23 Fall-Down macros, and tests/ fully aligned under Rev 5, the immediate priority became documenting this exact architecture through live code snapshots and clean multi-volume PDFs (DevLog 020) before advancing Block 3 audits into core/, audio/, and viewport/.

Navigation: ← DevLog 018 | Nov 2026 DevLog Index | Next: DevLog 020 →

Chapter 19. DevLog 020: 5-Block AsciiDoc, Live Code Snapshots & Paired Textbooks

Source File:
docs/dev_logs/2026/11/architecture/devlog_020_five_block_asciidoc_snapshots_pdf_budgets_and_paired_docs.adoc

> **Volume VI — Engineering Chronology (DevLog 020)**
> **Milestone Date:** 2026-11-12 | **Scope:** tools/src/guide.rs, docs/adoc_pdf_layout_rules.json, docs/code/{log,debug,io}/, docs/architecture/chronicles/, docs/offline_work/, Volumes I–VI
> **Series Navigation:** ← DevLog 019 | Nov 2026 Index | **(Latest DevLog)**
>
> Inspecting generated PDF output—especially **Volume V** (VOLUME_V_CORE_ENGINE_CODE_TOURS.pdf)—revealed awkward whitespace pages, broken [abstract] chapter numbering in Asciidoctor book builds, duplicate snapshot plate anchors, and stale manual prose drifting away from live C source files. To turn documentation into a true data-driven textbook system, master_guide gained live 5-mode code snapshots and PDF layout validation while docs/code/{log,debug,io}/, docs/architecture/chronicles/, and docs/offline_work/ were rebuilt onto the **Universal 5-Block AsciiDoc Standard**.

19.1. Textbook-Grade Live Code Snapshots (docs/code_snapshots_manifest.json)

Copying C functions by hand into documentation guarantees bit-rot. Extending tools/src/guide.rs introduced a declarative extraction manifest (docs/code_snapshots_manifest.json) supporting five surgical capture modes:

Snapshot Mode	Extraction Behavior & Textbook Usage
whole_file	Extracts a clean compact file wholesale (such as cmake/GenerateConfigRom.cmake or core test headers).
block	Captures an entire OmniCode BLOCK 1..BLOCK 5 section from .h or .c.
sub_assembly	Extracts an exact Sub-Assembly N.M region (e.g., io/ 5-block getters or debug/ probe structs).
function	Isolates an individual C function body with its paired dual-audience docstring.
symbol_group	Collects related type declarations or helper families into one compact visual plate.

- **Anchor Deduplication (deduplicate_snapshot_plate_anchors):** Eliminated duplicate [[snap_plate_*]] Asciidoctor anchor collisions whenever a snapshot appeared inline inside a chapter and again in a volume appendix.
- **Out-of-Box Theme & Title Formatting (docs/guide_theme_config.json, docs/themes/):** Cleaned Volume V synopsis whitespace so cover → TOC → overview → paired chapters flow cleanly across A4 pages without empty half-page breaks.

19.2. The 5-Block AsciiDoc Comment Standard & --validate-layout

Codified docs/adoc_5block_standard.json and docs/adoc_pdf_layout_rules.json so AsciiDoc manuscripts follow the exact same **Universal 5-Block Lifecycle Protocol** as .c, .h, and .json files:

- //// BLOCK 1..5 //// + // --- Sub-Assembly X.Y: AsciiDoc block/line comments frame Metadata, Opening, Body, Audit/Verification, and Closing/Navigation without rendering noise in HTML/PDF outputs.

- **PDF Hygiene Enforcement (`master_guide -- --validate-layout`):** Audits line-width budgets (`⇐ 104` cols with line numbers so Rouge blocks never soft-wrap in A4 PDFs), bans `[abstract]` above `==` headings, bans page-breaking `'''` horizontal rules in multi-chapter books, and enforces 6-hyphen `------` fences.

19.3. Paired Code Folders, Architectural Chronicles & Linked Offline Series

- `docs/code/{log,debug,io}/` **Paired Documentation:** Organized each root subsystem into its own folder pairing every `.h` and `.c` file into unified module manuals (plus a dedicated testing chapter per folder covering unit, functional, integrated, and system tiers).
- `docs/architecture/chronicles/`**:** Added accessible, mixed-audience deep dives explaining **why** the Rev 5 architecture was built for long-term game payoff (`01_root_stratum_story_and_game_payoff.adoc`) and how `AGENTS.md` evolved alongside AI-assisted engineering (`02_agents_md_evolution_and_ai_architecture.adoc`).
- `docs/offline_work/` **Series Unification (Parts 1–6):** Moved `Offline_Work_5.md` and `Offline_Work_6.md` out of repo root into `docs/offline_work/`, cleaned corrupted table math delimiters and citation tags in Part 6, authored `Offline-Work-5.adoc` + `Offline-Work-6.adoc`, upgraded `README.adoc` and `Offline-Work-1..4.{adoc,md}` into 5-block format with bidirectional navigation, and added `docs/offline_work/` to strict layout validation.

19.4. Verification Summary Across Code, Layout & Books

- **Native Rust Tooling (`RUSTFLAGS="-D warnings" cargo test`):** Verified snapshot extraction, anchor deduplication, layout rule checks, and validator modules cleanly with zero compiler warnings.
- **PDF Layout Validator (`master_guide -- --validate-layout`):** Enforced zero errors across strict targets (`docs/README.adoc`, `docs/architecture/`, `docs/code/{log,debug,io}/`, `docs/offline_work/`).
- **Full Book Compilation (`master_guide -- --all`):** Compiled standalone **Volumes I–VI** alongside the complete `MASTER_GUIDE.pdf` with DevLogs `001`–`020` indexed cleanly in Volume VI.

19.5. Synchronizing the Repository & Preparing for `core/` Audits

With code, tests, Config ROMs, snapshot books, pre-production chronicles, and DevLogs `016`–`020` all aligned under **Rev 5**, the workspace is ready for synchronized wave commits under `.githooks/commit-msg` before beginning the Block 3 audit across `core/`, `audio/`, `viewport/`, and `game/`.

Navigation: ← DevLog 019 | Nov 2026 DevLog Index | Documentation Root

Chapter 20. Developer Log 002: The Sacyra Chronicles, Dual Protagonists & TRPG Evolution

Source File: `docs/dev_logs/2026/10/lore_and_design/devlog_002_the_sacyra_chronicles_and_trpg_vision.adoc`

20.1. Abstract

This developer log chronicles the foundational provenance and overarching creative vision of **Tenebris and Lux**, rooted in its 2014–2018 high school inception as the premier game project of CreativeWorkzStudio. It documents the elimination of placeholder factions in favor of the canonical **Kingdom of Sacyra**, the introduction of dual protagonists **Elijah Ro-enheart** and **Katrina Jaren**, and the technical specification of the game's true genre: a **Tactical Strategy RPG (TRPG)** fusing grid-based tactical maneuvering with cinematic traditional side-scroll / action-based combat resolution.

20.2. Historical Provenance: The 2014–2018 High School Genesis

Before CreativeWorkzStudio evolved into a multifaceted publishing house, **Tenebris and Lux** was its very first major game in active development.

- **The RPG Maker Era (2014–2018)**: Conceived while the author was in high school learning game design and programming. The initial iterations were built using the default offerings of RPG Maker, exploring community plugins, and crafting characters through its Character Generator.
- **The Lost Assets & The Blank Canvas**: With no legacy digital assets surviving from that era, the project stands not as a nostalgic port, but as a complete ground-up rebirth—realizing the original creative vision with the systems engineering maturity, mathematical rigor, and architectural discipline of modern custom engine design.

20.3. Narrative Realignment: The Kingdom of Sacyra

During early prototype scaffolding, generic placeholder factions ("The Solar Order" and "The Umbral Covenant") were inserted to test basic moral sway logic. On October 26, 2026, the creator firmly realigned the canon, stripping away these synthetic factions to reveal the true world:

20.3.1. The Kingdom of Sacyra (`/s□□□ki□r□/`)

- The setting is the historic **Kingdom of Sacyra** (pronounced **"saw-kee-ruh"**).
- Sacyra is a structured, chivalric realm anchored by three distinct socio-political pillars:
  1. **The Crown & Royal Guard**: Guardians of legal order, national defense, and the ancestral throne.
  2. **The Noble Houses & Royal Court**: High aristocracy governing regional fiefs and vying for political dominance on the royal council.
  3. **The Adventurers' Guild**: A pragmatic, frontier-based brotherhood operating beyond courtly vanity, answering commoner monster distress calls and exploring untamed ruins.
- **The Existential Threat**: The ancient sovereign **The Dark Lord** breaking through the Abyssal Vaults, threatening to extinguish all mortal agency.

20.3.2. The Dual Protagonists

Rather than a generic lone wanderer, the narrative is anchored by two contrasting protagonists whose fates, lineages, and martial philosophies intertwine:

Character	Elijah Ro-enheart	Lady Katrina Jaren
Social Origin	Military Lineage (Common/Barracks Nobility)	High Aristocracy (House Jaren)
Family Heritage	Son of the former Head of the Royal Guard	Daughter of an influential noble house on the Royal Court
Internal Desire	Living up to his father's legendary military legacy while discovering what true honor demands	Escaping stifling aristocratic court expectations to join the local **Adventurers' Guild**
Relic Synergy	**The Shield of Lux** (Sanctuary, Warding, Formation Hold)	**The Sword of Tenebris** (Severance, Mist-Veil, Critical Strike)
TRPG Doctrine	Frontline heavy vanguard; zone-of-control denial; ally interception	Agile outrider skirmisher; bypassing enemy lines; flanking assassin

20.4. Ludological Evolution: The Tactical Strategy RPG (TRPG)

The game's true mechanical identity is a **Tactical Strategy RPG (TRPG)**. This fundamentally informs how the simulation engine and presentation viewports are being constructed:

20.4.1. The Dual-Layer Combat Loop

The combat loop operates across two synchronized phases:

1. **Strategic Grid Phase**:
   - The battle takes place on a discrete 2D grid representing castles, ramparts, thickets, and void-blighted frontier passes.
   - Units position based on movement ranges, terrain defense bonuses, directional facing (Front, Flank, Rear), and Zone-of-Control (ZOC) restrictions.
2. **Cinematic Combat Resolution (Side-View Action)**:
   - When a unit initiates combat against an enemy target, the game transitions into a dynamic **2D traditional side-scroll / action-based clash**.
   - Attacks play out with timed player agency: precision blocks with the Shield of Lux, poise-breaking shield bashes, and devastating severance slashes with the Sword of Tenebris.
   - Upon duel conclusion, remaining unit health, stagger, and relic resonance seamlessly persist back to the tactical grid.

20.4.2. Architectural Implications for the C Engine

Knowing this was originally an RPG Maker project gives us clear architectural directives: * **Decoupled Data Architecture**: Static definitions (`Units`, `Skills`, `Equipment`, `TerrainTypes`) must exist as clean data tables distinct from session state (`player_state_t`, `grid_t`). * **Deterministic Event Pipeline**: Interactive map features (gates, switches, dialog triggers) will follow a clean, reentrant command interpreter model. * **Zero Engine Bloat**: By building this custom in C11 rather than relying on bloated runtimes, our TRPG will maintain sub-millisecond turn updates, instantaneous scene transitions, and absolute softlock immunity.

20.5. Next Milestone Roadmap

1. **Phase 1 (Complete)**: Established OmniCode 5-block layering, Layer 0 utilities, CTest verification suite, and canonical documentation.
2. **Phase 2 (Active)**: Expand simulation state to model dual protagonists (Elijah & Katrina), party data structures, and the foundational 2D tactical grid.
3. **Phase 3**: Implement the cinematic side-scroll combat transition in the presentation layer.
4. **Phase 4**: Expand the tactical campaign across the provinces of Sacyra to confront the vanguard of The Dark Lord.

Appendix A: Wholesale Source Code Reference Plates

Complete verbatim reference plates for source files, blocks, sub-assemblies, and function groups excerpted in preceding chapters.

A.1. Pre-Flight Loose-Bolts Engine Walk (manifest_run_preflight)

Snapshot ID: snap_manifest_fn_preflight | **Mode**: Function | **File**: engine/src/core/manifest.c (L412–L509 of 669 lines) | Jump back to chapter text

```
412 /**
413  * @brief Executes comprehensive pre-flight diagnostics across all registered modules.
414  */
415 preflight_status_t manifest_run_preflight(manifest_registry_t *registry)
416 {
417     if (registry == NULL) {
418         debug_trap_tripwire("manifest", "NULL registry passed to manifest_run_preflight", __FILE__, __LINE__);
419         return PREFLIGHT_STATUS_HALT_FATAL;
420     }
421 
422     registry->pass_count = 0;
423     registry->warn_count = 0;
424     registry->critical_count = 0;
425     registry->fatal_count = 0;
426 
427     /* Step 1: Active DAG Dependency Graph Verification */
428     for (size_t i = 0; i < registry->count; i++) {
429         const module_metadata_t *mod = registry->modules[i];
430         if (mod == NULL) continue;
431 
432         for (size_t d = 0; d < mod->dependency_count; d++) {
433             const char *req_dep = mod->dependencies[d];
434             if (req_dep == NULL) continue;
435 
436             bool found = false;
437             for (size_t j = 0; j < registry->count; j++) {
438                 if (registry->modules[j] != NULL &&
439                     strcmp(registry->modules[j]->module_id, req_dep) == 0) {
440                     found = true;
441                     break;
442                 }
443             }
444 
445             if (!found) {
446                 diag_result_t dep_res;
447                 dep_res.passed = false;
448                 dep_res.severity = SEVERITY_FATAL;
449                 dep_res.subsystem = mod->module_id;
450                 dep_res.check_name = "Dependency DAG";
451                 dep_res.details =
452                     "Fatal: Unregistered upstream dependency declared.";
453                 registry->results[i] = dep_res;
454                 registry->fatal_count++;
455                 registry->status = PREFLIGHT_STATUS_HALT_FATAL;
456                 debug_trap_tripwire("manifest", "Unregistered upstream DAG dependency detected", __FILE__, __LINE__);
457                 return registry->status;
458             }
459         }
460     }
461 
462     /* Step 2: Individual Module Pre-Flight Verification */
463     for (size_t i = 0; i < registry->count; i++) {
464         const module_metadata_t *mod = registry->modules[i];
465         if (mod == NULL) {
466             continue;
467         }
468 
469         diag_result_t res;
470         if (mod->preflight_check != NULL) {
471             res = mod->preflight_check();
```

```
        } else {
            res.passed = true;
            res.severity = SEVERITY_PASS;
            res.subsystem = mod->module_id;
            res.check_name = "Presence";
            res.details = "No active preflight check defined.";
        }

        registry->results[i] = res;

        bool is_fatal = (res.severity == SEVERITY_FATAL) ||
            (!res.passed && res.severity != SEVERITY_WARNING &&
             res.severity != SEVERITY_CRITICAL);

        if (is_fatal) {
            registry->fatal_count++;
            registry->status = PREFLIGHT_STATUS_HALT_FATAL;
            return registry->status;
        } else if (res.severity == SEVERITY_CRITICAL) {
            registry->critical_count++;
        } else if (res.severity == SEVERITY_WARNING) {
            registry->warn_count++;
        } else {
            registry->pass_count++;
        }
    }

    /* Step 3: Determine aggregate system status */
    if (registry->critical_count > 0) {
        registry->status = PREFLIGHT_STATUS_DOWNGRADED;
    } else if (registry->warn_count > 0) {
        registry->status = PREFLIGHT_STATUS_WARNINGS;
    } else {
        registry->status = PREFLIGHT_STATUS_OK;
    }

    return registry->status;
}
```